Key

Style 5

- Truth and Nil 6
- Arithmetic 16
- Constants and Chance 38
- Number Theory 49
- Polynomials and Roots 57
- Matrices 64
- Arrays 72
- Boxes 92
- Sets and Membership 104
- Trains 114
- Functions 134
- Errors 147
- Classes and Properties 158
- The Virtual 169

Dungeon 175

- Places 176
- Classes 180
- Objects 186
- Entities 191
- Food 194
- Species inheritance 213

Appendix: Sugars and Exchanges 253

Appendix: Back Rank 258

Appendix: Persons 261

Appendix: Evaluation Order 265

Contents

Style 5

Truth and Nil 6

Niladic 6

Monadic 8

Dyadic 10

Arithmetic 16

Niladic 16

Monadic 20

Dyadic 31

Constants and Chance 38

Niladic 38

Monadic 43

Dyadic 46

Number Theory 49

Monadic 49

Dyadic 52

Polynomials and Roots 57

Monadic 57

Dyadic 60

Matrices 64

Niladic 64
Monadic 66
Dyadic 69
Arrays 72
Niladic 72
Monadic 75
Dyadic 83
Boxes 92
Niladic 92
Monadic 94
Dyadic 98
Sets and Membership 104
Niladic 104
Monadic 106
Dyadic 109
Trains 114
Niladic 114
Monadic 118
Dyadic 131
Functions 134
Niladic 134
Monadic 136
Dyadic 142
Errors 147
Niladic 147
Monadic 153
Classes and Properties 158
Niladic 158

Monadic 162
Dyadic 165
The Virtual 169
Niladic 169
Monadic 172
Dungeon 175
Places 176
Classes 180
Objects 186
Entities 191
Food 194
Species inheritance 213
Appendix: Sugars and Exchanges 253
Appendix: Back Rank 258
Appendix: Persons 261
Appendix: Evaluation Order 265
Appendix: Extending Wuxia 269
Index 273
English Index 274
Glyph Index 281

Style

Truth and Nil: Niladic

Wuxia

俠

The Hero

U+4FA0

Nil



The empty set — quiet failure value; absorbs; Nil⊡function → Nil

U+2205

Truth

⊤

Verum — first-class truth; no true anymore

U+22A4

Truth and Nil: Monadic

Invert

~

Invert — monadic only (tilde). Scalar: ~ 0 → 1, ~ 1 → 0 (user wording to confirm). Unary scalar verbs operate on array back ranks (new 2026-10-11)

U+007E

Not

¬

Not — monadic boolean not: ¬ Truth yields Nil, ¬ Nil yields Truth (new 2026-10-14)

U+00AC

Congruent

≡

Ambivalent: monadic binds train to lhs⊡function

U+2261

Truth and Nil: Dyadic

Less Than



Less Than — dyadic < A B yields Truth when A < B, Nil otherwise (new 2026-10-14)

U+003C

Match



Match — dyadic ≗ A B yields Truth when A and B are identical in shape, depth, values, and type, recursively throughout; Nil otherwise (Dyalog's dyadic ≡; glyph reassigned from Unbox 2026-10-20, Unbox remaining at ⊐)

U+2257

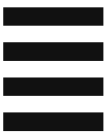
Congruent



Ambivalent: dyadic ≡ A B yields Truth when A and B have the same binding (same location in the system tables), Nil otherwise

U+2261

IsAt



IsAt — dyadic ≣ X Y yields Truth when X and Y share the same pointer address, Nil otherwise (new 2026-10-14)

U+2263

Equality



Equality — dyadic = A B yields Truth when A = B, Nil otherwise (new 2026-10-14)

U+003D

Greater Than



Greater Than — dyadic > A B yields Truth when A > B, Nil otherwise (new 2026-10-14)

U+003E

Less Or Equal



Less Or Equal — dyadic ≤ A B yields Truth when A ≤ B, Nil otherwise (new 2026-10-14)

U+2264

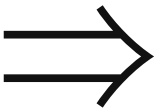
Greater Or Equal



Greater Or Equal — dyadic ≥ A B yields Truth when A ≥ B, Nil otherwise (new 2026-10-14)

U+2265

Implies



Implies — logical implication; ∅ ⇒ ∅ → ⊤

U+21D2

Approx



Approx — dyadic ≈ A B yields Truth when|A - B|is within 1e-6, Nil otherwise (new 2026-10-14)

U+2248

UnApprox



UnApprox — dyadic ≉ A B: sugar for ¬ ≈ A B (boolean Not of Approx) (new 2026-10-14)

U+2249

Unequal



Unequal — dyadic ≠ A B: sugar for ¬ = A B (boolean Not of Equality) (new 2026-10-14)

U+2260

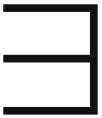
forall



Train quantifier — every element must evaluate non-nil. Idiom: dyadic NAND is ¬ ∀ 𝕀𝕀 X Y

U+2200

Forsome



Existential — at least one element evaluates non-nil

U+2203

Fornone



Not-forsome — defined as ~Forsome

U+2204

Arithmetic: Niladic

Signum / Plus

+

Ambivalent: Niladic + yields 0.
(Was ∓ U+2213, retired 2026-10-12

U+002B

Negate / Subtract

−

Niladic - yields 0 (new 2026-10-11

U+002D

Plus (Complex)

⨢

Plus (Complex), aka Sum Complex, Add Complex —; Niladic: sugar for ℿ 0 0, the complex zero (new 2026-10-14)

U+2A22

Times (Complex)

⨣

Times (Complex) —; Niladic: sugar for ℿ 1 0, the complex one (new 2026-10-14)

U+2A30

Minus (Complex)



Minus (Complex) —; Niladic: sugar for ⨿ 0 0, the complex zero (new 2026-10-14)

U+235B

Times (Polar)



Times (Polar) —; Niladic: the polar one, 2×1 [1; 0] (new 2026-10-14)

U+2A02

Divide (Complex)



Divide (Complex) —; Niladic: sugar for ⨿ 1 0, the complex one (new 2026-10-14)

U+2A2C

Divide (Polar)



Divide (Polar) —; Niladic: the polar one, 2×1 [1; 0] (new 2026-10-14)

U+2A38

Factorial

!

Factorial —; Niladic ! yields 1 (new 2026-10-12)

U+0021

Percent

%

Percent —; Niladic % is 0 (new 2026-10-12

U+0025

Square



Square — ⊠ X is X squared.; Niladic ⊠ yields 1, derived from Multiply (new 2026-10-12)

U+22A0

Multiply



*Multiply —; Niladic ⨯ yields empty array. Replaces Star * U+002A (new 2026-10-12; boxed forms 2026-10-12*

U+2A2F

Arithmetic: Monadic

Signum

+

Ambivalent: monadic Signum (less than zero yields −1, greater than zero yields 1, zero yields 0; back-rank scalar unary)

U+002B

Negate

-

Negate (monadic - X, unary minus)

U+002D

Conjugate

⊶

Conjugate — monadic ⊶X: complex conjugate; negates the imaginary part, preserving orientation (on [r,i] yields [r,−i]; on [r;i] yields [r;−i]) (new 2026-10-15)

U+22B6

Double

+2

Double — monadic +⍨X yields 2 × X (new 2026-10-15)

U+2A27

Sum

∑

Sum — sugar for reduce-addition: ∑ X means / + X (redefined 2026-10-12)

U+2211

Square Root

√

Square Root — monadic √ X yields the square root of X (new 2026-10-15)

U+221A

Plus (Complex)

⨁

Plus (Complex), aka Sum Complex, Add Complex —; Monadic: if X is scalar, sugar for ℙ X 0 (promote to complex); else identity, yields X.

U+2A22

Times (Complex)

⨂

Times (Complex) —; Monadic: if X is scalar, sugar for ℙ X 0 (promote to complex); else identity, yields X.

U+2A30

Minus (Complex)



Minus (Complex) —; Monadic: if X is scalar, sugar for ℚ X 0 (promote to complex); else identity, yields X.

U+235B

Times (Polar)



Times (Polar) —; Monadic: if X is scalar, promote to polar 2×1; else identity, yields X.

U+2A02

Divide (Complex)



Divide (Complex) —; Monadic: if X is scalar, sugar for ℚ X 0 (promote to complex); else identity, yields X.

U+2A2C

Divide (Polar)



Divide (Polar) —; Monadic: if X is scalar, promote to polar 2×1; else identity, yields X.

U+2A38

Percent



Percent —; monadic % X is absolute value, a back-rank operator.; monadic redefined 2026-10-12)

U+0025

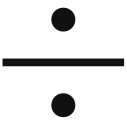
Star



*Star — exponentiate: monadic * X is 1, because the missing exponent Y is assumed 0, giving X^0 (new 2026-10-12). Idiom: X-th root of Y is * Y ÷ 1 X*

U+002A

Quotient



Quotient —; Monadic ÷ X: if X is a square array, yields the matrix inverse; else reciprocal, a back-rank scalar operator (new 2026-10-12)

U+00F7

Multiply



Multiply —; Monadic × X yields X when X numeric; on boxed X, catenates once.; monadic redefined 2026-10-12)

U+2A2F

Ceiling



Ceiling —; monadic ⌈X rounds up
(from APL; new 2026-10-12)

U+2308

Floor



Floor —; monadic ⌊X rounds down
(from APL; new 2026-10-12)

U+230A

Sine



Sine (new 2026-10-12)

U+25DD

Cosine



Cosine (new 2026-10-12)

U+25DE

Arccosine



Arccosine (new 2026-10-12)

U+25F6

Arcsine



Arcsine (new 2026-10-12)

U+25F7

Tangent



Tangent (new 2026-10-12)

U+2AD7

Arctangent



Arctangent (new 2026-10-12)

U+2AD8

Hyperbolic Sine



Hyperbolic Sine (new 2026-10-12)

U+2AA6

Hyperbolic Cosine



Hyperbolic Cosine (new 2026-10-12)

U+2AA7

Hyperbolic Arcsine



Hyperbolic Arcsine (new 2026-10-12)

U+2ABD

Hyperbolic Tangent



Hyperbolic Tangent (new 2026-10-12)

U+2AD1

Natural Log



Natural Log —; monadic ⍁ X is ln X.; Monadic ⍁ on a polar 2×1 yields the rectangular complex pair [ln r, θ].

U+237C

Antilog



Antilog — sugar for Star: ⍋̇ X is e raised to the X. Monadic ⍋̇ on a rectangular complex pair yields the polar 2×1 [e^a; b] (new 2026-10-12; glyph was 𝒆 U+1D58A until 2026-10-14)

U+29CA

Hyperbolic Arctangent



Hyperbolic Arctangent (new 2026-10-12)

U+2A4D

Hyperbolic Arccosine



Hyperbolic Arccosine (new 2026-10-12)

U+2ABE

Factorial

!

Factorial —; monadic ! X is X factorial

U+0021

Fourier



Fourier — monadic F X: FFT of X (new 2026-10-15)

U+03DC

InverseFourier



InverseFourier — monadic ⅁ X: inverse FFT of X (new 2026-10-15)

U+2132

Base Ten Log



Base Ten Log — ⍙ X is log base 10 of X, sugar for ⍦ 10 X (new 2026-10-12)

U+2A5F

Magnitude



Magnitude — monadic ⍫X: absolute value of X (new 2026-10-16)

U+2357

Arithmetic: Dyadic

Plus



Ambivalent: dyadic + X Y yields X + Y (addition).; dyadic 2026-10-15)

U+002B

Subtract

Subtract (dyadic - X Y yields X - Y, first minus second), the subtract that works with the / right-fold Reduce.; dyadic 2026-10-12)

U+002D

Plus (Complex)



Plus (Complex), aka Sum Complex, Add Complex —; dyadic: sum of complex numbers (1×2 rectangular), per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before adding.

U+2A22

Times (Complex)



Times (Complex) —; dyadic: product of complex numbers, per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before multiplying.

U+2A30

Minus (Complex)



Minus (Complex) —; dyadic: difference of complex numbers, per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before subtracting.

U+235B

Times (Polar)



Times (Polar) —; dyadic: product of polar numbers; uses the 2×1 oriented vector to hold magnitude and direction: magnitudes multiply, angles add; if either or both X and Y are scalar, promote to polar 2×1 before multiplying.

U+2A02

Divide (Complex)



Divide (Complex) —; dyadic: quotient of complex numbers, per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before dividing.

U+2A2C

Divide (Polar)



Divide (Polar) —; dyadic: quotient of polar numbers; uses the 2×1 oriented vector to hold magnitude and direction: magnitudes divide, angles subtract; if either or both X and Y are scalar, promote to polar 2×1 before dividing.

U+2A38

Percent



Percent —; dyadic % X Y yields the remainder of X divided by Y

U+0025

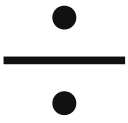
Star



*Star — exponentiate: dyadic * X Y is X to the power Y (left=base, right=exponent)*

U+002A

Quotient



Quotient —; dyadic ÷ X Y divides X by Y, yielding a real; division by zero yields infinity, except 0/0 which yields Nil.

U+00F7

Multiply



Multiply —; dyadic multiplies its arguments; when X is boxed, ⍪X Y catenates X repeated Y times.

U+2A2F

Factorial

!

Factorial —; dyadic ! X Y is the binomial coefficient Y choose X.

U+0021

Ceiling

⌈

Ceiling —; dyadic ⌈ X Y is maximum

U+2308

Floor

⌊

Floor —; dyadic ⌊ X Y is minimum

U+230A

Natural Log



Natural Log —; Dyadic ⍟ X Y is log base X of Y, sugar for ÷ ⍟ Y ⍟ X (new 2026-10-12)

U+237C

Square



Square — ⊠ X is X squared.; Dyadic ⊠ X Y squares X Y times.

U+22A0

Constants and Chance: Niladic

Half

½

Half —; niladic: 0

U+00BD

Time



Time — an IO intrinsic. The underlying mathematical event: its value is computer era plus microseconds. Every event is a read at this coordinate, and the reading never comes back Nil (2026-10-20)

U+231A

Bell

𝔅

Bell —; niladic: random point in (-1, 1), exclusive

U+1D56D

Real

ℜ

Real — niladic: random real number between 0 and 1; sugar for ½ + 1 𝔅 (new 2026-10-13)

U+1D57D

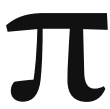
Gamma



Gamma — niladic γ: Euler-Mascheroni constant (new 2026-10-14)

U+03B3

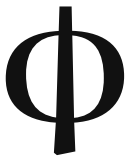
Pi



Pi — niladic π: value of pi (new 2026-10-14)

U+03C0

Golden



Golden — niladic φ: golden mean (new 2026-10-14)

U+03C6

Euler



Euler — niladic ℇ: Euler's constant (new 2026-10-14)

U+2107

D2



D2 — niladic: a single 2-sided die (1-2); sugar for 𝔇2; the die face U+2681 (2026-10-20)

U+2681

D4



D4 — niladic: a single 4-sided die (1-4); sugar for 𝔇4; the die face U+2683 (2026-10-20)

U+2683

D6



D6 — niladic: a single 6-sided die (1-6); sugar for 𝔇6; the die face U+2685 (2026-10-20)

U+2685

Dice



Dice —; niladic: single 6-sided die (1-6)

U+1D56F

Dice Percentile

‰

*Dice Percentile — niladic, intrinsic: A gets D10, B gets D10, yields + B * 10 A; the per mille sign U+2030 (2026-10-20)*

U+2030

D10



D10 — niladic: a single 10-sided die (1-10); sugar for 𝔇10; the circled ten U+2469 (2026-10-20)

U+2469

D12



D12 — niladic: a single 12-sided die (1-12); sugar for 𝔇12; the circled twelve U+246B (2026-10-20)

U+246B

Pick



Pick —; niladic: Nil

U+2A03

Constants and Chance: Monadic

Half

½

Half —; monadic ½ X: X divided by 2; array → sugar for ⊞(⍴ X); train → floor(length/2) index; dictionary → Nil (new 2026-10-13)

U+00BD

Pick

⍤

Pick —; monadic ⍤X: scalar → same scalar; train → use Dice of size train to select element; array → use Dice of sides from shape to select element; dictionary → random {key:value} pair (new 2026-10-13)

U+2A03

Bell

𝔅

Bell —; monadic 𝔅μ: bell curve point with mean μ (μ in (-1,1))

U+1D56D

Dice

𝔇

Dice —; monadic 𝔇X: single X-sided die (1-X)

U+1D56F

Deal



Deal — given a scalar, works as expected; given an array, fills the entire back rank with limited values. Concrete examples and summary TBD (new 2026-10-11)

U+003F

Constants and Chance: Dyadic

Bell

𝔅

Bell —; dyadic 𝔅 μ σ: bell curve point with mean μ and stddev σ (σ ≤ 1/3) (new 2026-10-13)

U+1D56D

Dice

𝔇

Dice —; dyadic 𝔇 X N: N X-sided dice; all sugar via ℜ (hence 𝔅) (new 2026-10-13; die face U+2685 tried and reverted 2026-10-20)

U+1D56F

Number Theory: Monadic

Divisible



Divisible —; monadic ⍱ X yields Truth when X is divisible by 2, Nil otherwise

U+23A2

Indivisible



Indivisible —; monadic ⍲ X: sugar for ¬ ⍱ X (the Odd test)

U+2AEE

Primes



Primes — monadic 𝒫X yields the prime factors of X, per J's q:. (new 2026-10-18)

U+1D4AB

Prime



Prime — monadic 𝔓 N yields the Nth prime, per J's p: (zero-indexed: 𝔓 0 is 2). (new 2026-10-18)

U+1D513

Prime



Prime — monadic ⌽ X: sugar for ⍵⍵/ X ⍳2 ⌊√ X ⍸ ≤ X 1 (new 2026-10-15)

U+23C0

Factor



Factor — factorization verb (new 2026-10-16)

U+27D2

Combinatoric



Combinatoric —; monadic ## X: factorial of X

U+2A69

Number Theory: Dyadic

LCM



LCM — dyadic ⍜ X Y: least common multiple of X and Y (new 2026-10-16)

U+238A

Divisible



Divisible —; dyadic ⍱ X Y yields Truth when X is divisible by Y, Nil otherwise (new 2026-10-15)

U+23A2

GCD



GCD — dyadic ⫻ X Y: greatest common divisor of X and Y (new 2026-10-16)

U+2980

Indivisible



Indivisible —; dyadic ⍲ X Y: sugar for ¬ ⍱ X Y (boolean Not of Divisible) (new 2026-10-15)

U+2AEE

Combinatoric



Combinatoric —; dyadic ⌗ X Y: binomial X choose Y (new 2026-10-16)

U+2A69

Encode



55

U+2AE7

Decode



Decode — dyadic B ≐ D: the inverse of Encode; recombines digits D in the radix system B into their value, e.g. 24 60 60 ≐ 6 30 56 is 23456. Dyalog's ⊥ under a Wuxia glyph (new 2026-10-21)

U+2AE8



Encode

U+2AE7

Encode — dyadic B ⍒ N: B is a vector of radices defining a number system; yields the digits of N in that system, one digit per radix, remainder and quotient from the right; too few radices truncate the high digits, and a leading 0 radix makes the first digit unbounded. Dyalog's ⊤ under a Wuxia glyph, since ⊤ is Truth here (new 2026-10-21)

Polynomials and Roots: Monadic

NextLocal

ℓ

NextLocal — the Roots division step: given P, yields [R prime, P prime] where R prime is Local(P) and P prime is P deflated by synthetic division of R prime. (new 2026-10-19)

U+2113

Roots

℞

Roots — sugar: ℞ P is ℜ ℓ P (Reveal NextLocal over P); yields all polynomial roots. (new 2026-10-18; factored via Reveal 2026-10-19)

U+211E

Differentiate

∂

Differentiate — train verb ∂ T yields symbolically differentiated version of train T of coefficients, multiplying constants as needed; same order as Polynomial (new 2026-10-13)

U+2202

Integrate

∫

Integrate — train verb ∫ T yields symbolically integrated version of train T of coefficients, dividing constants as needed; same order as Polynomial (new 2026-10-13)

U+222B

Solve

&

Solve — monadic & A: array → vector of polynomial coefficients; missing input row is all zeroes (new 2026-10-13)

U+0026

Optimize



*Optimize —; monadic 𝔅 V: vector → polynomial; searches free variable with delta = 0.01 * domain, epsilon defaults to 1e-6*

U+214B

Local



Local — monadic ⍵ V is 𝔅 ⫴ℒ & V (FixedPoint Optimize Triple Solve V); approximates best x for a local minimal of V (new 2026-10-13)

U+1D50F

Polynomials and Roots: Dyadic

Reveal



Reveal — intrinsic. Dyadic ℜ P Y yields the successive application of P upon Y as Y shrinks to Nil. Epsilon from P has two elements: an alpha to apply P to next Y, and the next Y. (new 2026-10-18)

U+211C

Optimize



Optimize —; dyadic 𝔅 V ε: recurse into smaller domains until change in c < ε (new 2026-10-13)

U+214B

Integer Division



Integer Division — dyadic ⫽ X Y divides X by Y, yielding an integer (replaces Slash / integer division from manual p.19; / is now Reduce). Vector form: ⫽ V X where V is vector yields synthetic quotient of V by X (new 2026-10-14)

U+2AFD

Evaluate



Evaluate — dyadic 𝒇X T evaluates train T of coefficients upon X (new 2026-10-13)

U+1D58B

FixedPoint



Fixed point — dyadic cø V guess: refine guess until epsilon < 1e-6 (max 100 steps); failure to converge returns infinity (new 2026-10-13)

U+AB44

Matrices: Niladic

Outer Product



Outer Product —; Niladic ⊗ yields 1 (new 2026-10-12)

U+2297

Inner Product



Inner Product —; Niladic · yields 1 (new 2026-10-12; matrix case 2026-10-14)

U+22C5

Multiply Matrices



Multiply Matrices —; Niladic ⋈ yields the 1×1 identity matrix (new 2026-10-14)

U+22C8

Matrices: Monadic

Outer Product



Outer Product —; Monadic ⊗ X yields X.

U+2297

Inner Product



Inner Product —; Monadic ⋅ X yields X.

U+22C5

Multiply Matrices



Multiply Matrices —; Monadic ⋈ X yields X.

U+22C8

L Factor



L Factor — monadic: the L (lower triangular, 1s on diagonal) factor of the LU decomposition of a rank-2 array (new 2026-10-14)

U+1D577

Determinant



Determinant — monadic ⌹X: determinant of square matrix X (new 2026-10-16)

U+233C

U Factor



U Factor — monadic: the U (upper triangular) factor of the LU decomposition of a rank-2 array (new 2026-10-14)

U+1D580

Matrices: Dyadic

Outer Product



Outer Product —; dyadic ⊗ X Y yields the outer product matrix where entry [i,j] is ⍺ X[i] Y[j]; shape is the concatenation of the shapes of X and Y.

U+2297

Inner Product



Inner Product —; dyadic ⋅ X Y: for vectors X, Y, the dot product (sum of pairwise products); for matrices, the Frobenius inner product: a scalar, the sum of all pairwise products.

U+22C5

Multiply Matrices



Multiply Matrices —; dyadic ⋈ X Y: matrix multiplication, (⋈ X Y)[i;j] = ⋅ X[i;] (⍤ Y)[j;]; inner dimensions must agree, non-square fine.

U+22C8

Divide Matrix



Divide Matrix — dyadic ⌹ X Y: sugar for ⋅ X ÷Y, Inner Product of X with the inverse of Y (monadic ÷ inverts a square array); solves the square system, no pseudoinverse. Dyalog's Matrix Divide glyph, taken as-is (new 2026-10-21)

U+2339

Arrays: Niladic

Pee

ℙ

Pee — Where⊡Pee yields Where [0, 1
(first two coordinates)

U+2119

Zee

ℤ

Zee — the last position of a slice
(Where⊡Zee yields Where [-1)

U+2124

Exx

𝕏

Exx — the zero position of a slice
(Where [0)

U+1D54F

Wye

𝕐

Wye — the midposition of a slice

U+1D550

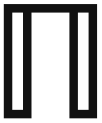
Extend



Extend —; niladic ⍐: sugar for ⊂ Nil

U+213E

Catenate



Catenate —; niladic ⍪: sugar for ⊂ Nil

U+213F

Grade Up



Grade Up — ⍋X yields the indices that sort X ascending. Niladic ⍋ yields 0 (from APL; new 2026-10-12)

U+234B

Grade Down



Grade Down — ⍒X yields the indices that sort X descending. Niladic ⍒ yields Nil (from APL; new 2026-10-12)

U+2352

Arrays: Monadic

Rank



Rank — monadic ≠ X yields the number of dimensions in X; zero when scalar. (new 2026-10-18)

U+22D5

Permute



Permute —; monadic ⍤ X and dyadic ⍤ X Y work as in J's C.. Both modes intrinsic. (new 2026-10-19)

U+2A0A

Laminate



Laminate —; monadic ⍪ X: adds a leading singleton axis

U+2A5E

Anagram



Anagram — permutation index/parity (J's A.). Both modes intrinsic. (new 2026-10-19)

U+1D49C

Enumerate

№

Enumerate — monadic № X: sugar for ⍳≢X ⍝ ⍴X; yields index-value pairs for X (new 2026-10-18)

U+2116

Reflect



Reflect — monadic ÷X: sugar for ⍳≢⌽- ⌽∘⍳X (new 2026-10-15)

U+223B

Count

78

⍳

U+2373

Kount



Kount — monadic ⍭B: takes a vector of T or Nil; returns a vector counting the T, turning each Nil to 0. Intrinsic. (new 2026-10-18)

U+2A23

⍳ Count

U+2373

Count — magical: when given a shape, fills an array matching shape. Scalar: behaves as normal. Array: fills array with next dimension limited by scalar from back rank — e.g. ⍳ 2 3 → rows of 0 1 2 (origin 0; last axis counts, outer axes replicate) (new 2026-10-11)

Mix



Mix — monadic ⥣ X yields an array formed from the items of X, opened one level and stacked along a new first axis: a vector of k items each of shape S yields shape k,S; mismatched items pad with fill (Dyalog's monadic ↑; new 2026-10-21)

U+21D1

Depth



Depth — monadic ⍤ X yields the nesting depth of X: a scalar has depth 0, an array of scalars has depth 1, and each further layer of nesting or boxing adds one, the deepest chain governing: Dyalog's definition of monadic ≡, on a Wuxia glyph (new 2026-10-20)

U+228D

Reverse



Reverse —; monadic ⌽ X reverses elements along the last axis

U+233D

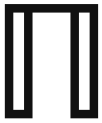
Transpose



Transpose —; monadic ⍉ X reverses the order of axes.

U+2349

Catenate



Catenate —; monadic ⎍ X: catenates X to itself

U+213F

Split



Split — monadic ⇊ X yields a vector of the items of X along its first axis, each item having the shape of X without its first element; the inverse of Mix (Dyalog's monadic ↓; new 2026-10-21)

U+21D3

RavelRows



RavelRows — monadic ⍪∈X: yields the rows of X as a vector (Antitable; new 2026-10-15)

U+22F2

Ravel



Ravel — monadic ∈⍪X: flattens array X to vector, row-major (new 2026-10-14)

U+22F3

Shape

ρ

Shape —; monadic ρ X: shape of X; scalar → 0; array → vector of dimensions; train → [1, length of train]; dictionary → [1, pairs present]

U+03C1

Extend



Extend —; monadic ⥊ X: Table X as in APL, adds trailing axis of length 1

U+213E

Sight



Sight — takes 2D array, finds center, returns vector location of nearest non-zero scalar; ties pick one (new 2026-10-13)

U+2388

Rotate



Rotate —; monadic ⃠ X: rotate X toward origin by 1

U+29BD

Cardinality



Cardinality — "how big is this?"; scalar → -1; array → number of elements held; dictionary → number of pairs; train → length of train (new 2026-10-13)

U+2210

Center



Center — takes a shape, returns its integer halves expressed as a vector (e.g. [10, 6] → [5, 3]) (new 2026-10-13)

U+229E

Arrays: Dyadic

Put



Put — ⫤⊨ X P A: value at coordinate P of array A becomes X (new 2026-10-18)

U+27DA

Permute



Permute —

U+2A0A

Laminate



Laminate —; dyadic ⩓̄ A B: yields [A; B]; when A and B are both scalars, yields A B (new 2026-10-17)

U+2A5E

Anagram



Anagram — permutation index/parity (J's A.). Both modes intrinsic. (new 2026-10-19)

U+1D49C

Take



Take — ↑ X Y takes the first X elements of Y (last X if X is negative); X may be a vector for per-axis counts (from APL; new 2026-10-12)

U+2191

Drop



Drop — ↓ X Y drops the first X elements of Y (last X if X is negative); X may be a vector for per-axis counts (from APL; new 2026-10-12)

U+2193

Reverse



Reverse —; dyadic ⌽ X Y rotates Y by X positions (from APL; new 2026-10-12)

U+233D

Transpose

86



U+2349



Transpose

U+2349

Transpose —; Dyadic ⍉ P X: the axes of X are ordered by P: one entry per axis of X, axis numbers from 0, axis i of X becoming axis P[i] of the result; a repeated entry collapses its axes into a diagonal section, running to the shorter side (dyadic as specified in Dyalog, taken as it stands 2026-10-21; from APL; new 2026-10-12)

Extend



Extend —; dyadic ⟦ X N: extends by N axes (new 2026-10-14)

U+213E

Catenate



Catenate —; dyadic ⟦⟧ X Y: catenate X and Y as in APL; variadic ⟦⟧ A ... Z: catenate A through Z left to right (new 2026-10-14)

U+213F

Glean

88



U+27D7

Expand

89



U+1D570



Glean

`U+27D7`

Glean — dyadic ⟗ X Y: Y indexed as in Dyalog: X is the index specification, one index set per axis, their cross product selecting the subarray of Y; a single index on every axis yields the cell itself, rank dropped; axes X omits are taken whole. Dyalog's Index ⌷ (new 2026-10-21)



Expand

`U+1D570`

Expand — dyadic 𝔈 P X: expands X under pattern P: a 1 takes the next item of X, a 0 inserts a fill item (the prototype: 0 for numbers, blank for characters); integer pattern: positive n repeats the next item n times, negative inserts that many fill items. Stop ⋄ separates the arguments. Dyalog's \ as specified, under a Wuxia glyph (new 2026-10-21)

Shape

ρ

Shape —; dyadic ρ S X: reshape X to shape S (per APL) (new 2026-10-13)

U+03C1

Rotate



Rotate —; dyadic ⌽ X Y: rotate X by Y (Y positive toward origin, negative away) (new 2026-10-16)

U+29BD

Boxes: Niladic

Box



Box —; Niladic ⊂ yields Truth
(from APL Enclose; new 2026-10-12)

U+2282

Unbox



Unbox —; Niladic ⊃ yields Nil
(from APL Disclose; new 2026-10-12)

U+2283

Summarize



Summarize — the summary effect at full power. Array case: find the 0,0 corner; if another array, recurse into the subarray; if a scalar, yield the boxed scalar. Scalar case yields the boxed scalar; niladic ⊚ is the 1,1 vector containing zero.

U+229A

Boxes: Monadic

Box



Box — monadic ⍈ A: wraps A in a box so it travels as a single element (new 2026-10-17)

U+2256

Zip

96



U+22B0

Catalogue



Catalogue —; monadic ⊚ X: cartesian product of the boxed lists in vector X (new 2026-10-17)

U+2A37

Raze



Raze — monadic ⫻ A: sugar for Ravel Map Unbox A; opens every box and joins the contents into one vector (new 2026-10-17)

U+2AF5



Zip

`U+22B0`

Zip — takes a train of vectors and zips them: ⍭ V1 V2 ... Vn yields an m×n matrix (m = vector length, n = train length); the resulting width is the length of the train; a scalar in the train expands into a column to fit the others; a container in the train contributes only its origin scalar, recursively (the summary effect); mismatched lengths Nil-pad (consistent with Diction); zips along the back rank for higher-rank arrays; ⍭ 1 2 yields the column [1;2] (new 2026-10-12; recycles the retired AntiSignum glyph)

Box



Box —; monadic ⊂X encloses X into a scalar box.

U+2282

Unbox



Unbox —; monadic ⊃X yields X.

U+2283

Summarize



Summarize — the summary effect at full power. Array case: find the 0,0 corner; if another array, recurse into the subarray; if a scalar, yield the boxed scalar. Scalar case yields the boxed scalar

U+229A

Boxes: Dyadic

Summarize

100



U+229A

Fetch



Fetch — dyadic ⊸V X: applies V as search vector into X's items; follows path V down through nested boxes (new 2026-10-17)

U+22B8

Catalogue



Catalogue —; dyadic ⊗A B: yields [[A0,B0] ... [A1,B1]] ... [[AN,B0] ... [AN,BN]], the outer product matrix of pairs

U+2A37

Link



Link — dyadic ⫟X Y: sugar for Catenate Map Box X Y; boxes X and Y into a two-element list (new 2026-10-17)

U+2AE9



Summarize

U+229A

Summarize — the summary effect at full power. Array case: find the 0,0 corner; if another array, recurse into the subarray; if a scalar, yield the boxed scalar. Scalar case yields the boxed scalar; Dyadic D ⊚ A: D (Direction) is a rank-1 array of coordinates; summarization starts at A[D], then proceeds as monadic; a scalar Direction s means start at (s, 0); Zip summarizes each train element (new 2026-10-12)

Box



Box —; Dyadic ⊂ X Y encloses X by Y levels; Y=0 leaves X unchanged.

U+2282

Unbox



Unbox —; Dyadic ⊃ X Y picks element X from Y.

U+2283

Partition

102



U+2286



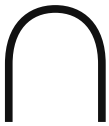
Partition

U+2286

Partition — dyadic ⊆ P X: partitions X into boxed segments under pattern P, one non-negative integer per item of X along its first axis: the first segment starts at the first item, a new segment starts wherever P rises, and items under a 0 in P are dropped. Dyalog's Partitioned Enclose, at Dyalog's own later glyph (new 2026-10-21)

Sets and Membership: Niladic

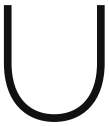
Intersection



Intersection —; niladic ∩: sugar for ⊂ Nil

U+2229

Union

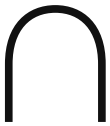


Union —; niladic ∪: sugar for ⊂ Nil

U+222A

Sets and Membership: Monadic

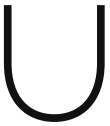
Intersection



Intersection —; monadic ∩ A: is A

U+2229

Union



Union —; monadic ∪X: unique as in APL

U+222A

InOf



InOf — the curry of In: ⇔InOf ⍷∈ X; monadic predicate testing membership in X. (new 2026-10-18)

U+22F8

Where



Where — monadic ⍸Y: returns vector of indices of all elements of array Y that are Truth (new 2026-10-18)

U+2A42

Enlist

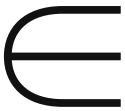


Enlist —; monadic ∈Y: as defined in APL (deep flatten to vector)

U+220A

Sets and Membership: Dyadic

In



In — dyadic ∈ Y X yields Truth when X is in Y, Nil otherwise (J's e.). (new 2026-10-18)

U+2208

InLast



InLast — dyadic ∈̇ X Y: yields the index of the last occurrence of X in Y; yields length of Y if not found (new 2026-10-18)

U+22F5

InAt



InAt — dyadic ⋵ X Y: yields the index of Y whose value equals X; yields length of Y if not found (new 2026-10-18)

U+22F9

Find



Find — dyadic P ⍷ X: yields a Boolean array of the same shape as X, Truth at the starting cell of each occurrence of the pattern P in X and Nil elsewhere; P is a whole sub-array, and occurrences may overlap. Dyalog's ⍷ imported as specified (new 2026-10-21)

U+2377

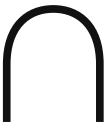
Bucket



Bucket — dyadic ⌶ X Y: yields index of interval in X that Y falls into (new 2026-10-18)

U+2210

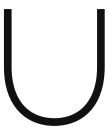
Intersection



Intersection —; dyadic ∩ A B: as in APL, elements in both (new 2026-10-14)

U+2229

Union



Union —; dyadic ∪ X Y: union as in APL (new 2026-10-14)

U+222A

Set Difference



Set Difference — dyadic ⍱ A B: elements in A but not in B (new 2026-10-14)

U+2A43

In



Enlist —; dyadic ∈ X Y:
find/membership as defined in APL
(new 2026-10-13)

U+220A

Trains: Niladic

Reduce



Reduce — / F V: start at the highest index (its scalar is the initial result), step down toward origin; at each step result = F(scalar, result). Right fold.; Niladic / returns 0 (new 2026-10-12)

U+002F

Map



Map — ⍬ A B executes function A with zero arguments upon all scalars of B, as a back rank operator; usual back scalar rules.; Niladic ⍬ yields Truth, same as niladic 𝔗 (new 2026-10-12)

U+226C

Reduce (forward)



⌿ F V: start at the origin (its scalar is the initial result), step up toward the highest index; at each step result = F(result, scalar). Left fold, the reversed scan.

U+233F

Identity



Identity — 𝔗 V yields V unchanged; on a train, yields the train unchanged. Niladic 𝔗 yields Truth (new 2026-10-12)

U+1D574

Backstroke



Backstroke — Scan: the running Reduce, paired with /. \ F V on a length-n V yields a length-n vector where element i is / F (right fold) applied to the first i elements of V.; Niladic \ yields 0 (new 2026-10-12)

U+005C

Stop



Stop — statement separator; separates statements on one line. A carriage return is sugar for Stop, and end of file is also sugar for Stop (2026-10-20)

U+22C4

Backstroke (forward)



Backstroke (forward) — Reverse Scan, paired with ⌿. ⍀ F V on a length-n V yields a length-n vector where element i is ⌿F (left fold) applied to the first i elements of V.

U+2340

Strip



Strip —; niladic: removes the train from head up to but not including the first Stop encountered

U+2ACF

Keyboard



Keyboard — an IO intrinsic; a blocking verb with no argument: the train waits inside it until a key lands, and it returns that key as a glyph (2026-10-20)

U+2328

Comment



Comment — disregards the entire following train, up to and including Stop (2026-10-20)

U+270C

Trains: Monadic

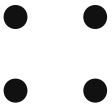
Meter

120



U+2214

Atomize



Atomize — monadic ∴A: reads up to the first 32 elements of A (origin to end) and prepares them for use by train elements ahead (new 2026-10-14)

U+2237

Duplicate



Duplicate — copies a train to a train twice as long: ⊷A B C → A B C A B C (was ==U+2A75, then ∴ U+2236 — settled 2026-10-12)

U+22B7

Caboose



Caboose — picks up the train from head to Stop, then converts boxed contents to a vector (inverse of Atomize) (new 2026-10-15)

U+2A78



Meter

U+2214

Meter — measures its argument: ∔ V is ≺ + ⊷ V (zip the needle with the signal); yields M, a sequence of (needle, signal) pairs, needle ∈ -/0/+. M works in place of V in all Collate modes: the needle selects the function, the function transforms the signal (was ±, corrected to ≺ 2026-10-12)

Reduce



Reduce — / F V: start at the highest index (its scalar is the initial result), step down toward origin; at each step result = F(scalar, result). Right fold.; Monadic / V yields a linear array of the scalars ripped from V, highest to lowest indices.

U+002F

Map



Map — ⍁ A B executes function A with zero arguments upon all scalars of B, as a back rank operator; usual back scalar rules.; Monadic ⍁ B is identical to Identity 𝔉 B.

U+226C

Reduce (forward)



⌿ F V: start at the origin (its scalar is the initial result), step up toward the highest index; at each step result = F(result, scalar). Left fold, the reversed scan.; Monadic ⌿ V and niladic ⌿ behave identically to / (new 2026-10-12)

U+233F

Zero



Zero — on a train, replaces all values with zeroes while keeping shapes (new 2026-10-12). As a noun, 𝟛 is the zero array (2026-10-20)

U+1D585

Backstroke

123



U+005C

Duplicate Head



Duplicate Head — ⅡA ... yields A A ...; duplicates head of train, aka duplicate/dup (new 2026-10-14)

U+2161

Backstroke (forward)



Backstroke (forward) — Reverse Scan, paired with ⍀. ⍀F V on a length-n V yields a length-n vector where element i is ⍀F (left fold) applied to the first i elements of V.; Monadic ⍀V and niladic ⍀ behave identically to \ (new 2026-10-12)

U+2340

Pipeline



Pipeline —

U+2AF7



Backstroke

`U+005C`

Backstroke — Scan: the running Reduce, paired with /. \ F V on a length-n V yields a length-n vector where element i is / F (right fold) applied to the first i elements of V.; Monadic \ V yields a linear array of the scalars ripped from V, lowest to highest indices.

Head



Head — sugar for ⊐1 (new 2026-10-15)

U+2991

Tail



Tail — sugar for ⊐-2 (new 2026-10-15)

U+2992

Duplicate Tail



Duplicate Tail — ⧟... A yields ... A A; duplicates tail of train (new 2026-10-13)

U+29DE

Strip

125



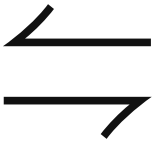
U+2ACF

⊐ Strip

U+2ACF

Strip —; monadic ⊐ X: removes the train from position X (0 is head) up to but not including the first Stop encountered; when X < 0, strips from position X in reverse order back to head, where −1 is the final tail position adjacent to the first Stop encountered (new 2026-10-15)

Swap



Swap — ⇋A B ... yields B A ...; swaps head two of train. Single: ⇋ A Stop X yields X. Bare: ⇋ Stop X yields X (new 2026-10-13)

U+21CB

Replicate

127



U+232C

Rotate Head



Rotate Head — ⟲A B C ... yields B C ... A; rotates head to tail (new 2026-10-13)

U+27F2

Rotate Tail



Rotate Tail — ⟳ ... A B C yields C ... A B; rotates tail to head (new 2026-10-13)

U+27F3



Replicate

U+232C

Replicate — ⌬ N X Stop yields X replicated N times; e.g. ⌬ 5 A Stop gives A A A A A. Zero case: ⌬ 0 A Stop X yields X. Negative N is antireplicate: ⌬ -1 A A A A A Stop yields A A A A. Empty antireplicate: ⌬ -1 Stop X yields X. Bare: ⌬ Stop X yields X (new 2026-10-13)

Nip



Nip — ↢ A B ... yields A ...; drops second, keeps head (new 2026-10-13)

U+21F7

Right



Right —; Monadic ⊢ X is Same: yields X unchanged (imported from Dyalog, 2026-10-21)

U+22A2

Left



Left —; Monadic ⊣ X is Same: yields X unchanged (imported from Dyalog, 2026-10-21)

U+22A3

Over



Over — ⍥ A B ... yields B A B ...; copies second to head (new 2026-10-13)

U+293A

Triple

⅀

Triple — ⅀ X yields X X X; triplicates top of stack (new 2026-10-13)

U+2162

Tuck



Tuck — ⇿ A B ... yields A B A ...; copies head under second (new 2026-10-13)

U+21F8

Reverse Train



Reverse Train — reverses order of train (new 2026-10-13)

U+21F9

Drop Train



Drop Train — sugar for reverse, replicate -1, reverse; drops head of train (new 2026-10-13)

U+291A

Fork

⫛

Fork — ⫛ F G H X: sugar for G H Swap F Dup X (J fork; new 2026-10-14)

U+22D4

Clock



Clock — an IO intrinsic; monadic blocking verb: Clock N blocks the train for N tenths of a second. It repeats: Clock is the event for People providing service, the heartbeat a service Person blocks on, wakes to serve, and blocks on again (2026-10-20)

U+23F1

Trains: Dyadic

Right



Right —; dyadic A ⊢ B yields B; A is evaluated and its result is discarded.

U+22A2

Left



Left —; dyadic A ⊣ B yields A; B is evaluated and its result is discarded.

U+22A3

Applyself



Applyself — ⍣ f n a applies f n times to a; sugar for ⋘a ⍵ ⊂ ⍵⍣f n (from APL power; new 2026-10-12)

U+2363

Pipeline



Pipeline —; dyadic ⋘X Y applies vector Y, readable as a train of monadic functions, in sequence to X (new 2026-10-12)

U+2AF7

Functions: Niladic

Unquote Triple



Unquote Triple — »: the closing mate of Quote Triple; it stays in the train, and its later execution transforms the parse buffer into the Temporary Vector ~~e~~, a temporary vector of characters (new 2026-10-21)

U+00BB

Unquote Single



Unquote Single — ’: the closing mate of Quote Single; it stays in the train, and its later execution transforms the parse buffer into the Temporary Vector ~~e~~, a temporary vector of characters (new 2026-10-21)

U+2019

Unquote Double



Unquote Double — ”: the closing mate of Quote Double; it stays in the train, and its later execution transforms the parse buffer into the Temporary Vector ~~e~~, a temporary vector of characters (new 2026-10-21)

U+201D

Functions: Monadic

Diction



Diction — dict-backed decision-table constructor (was Vectorize, renamed 2026-10-11). ¡ A B → {A:B}; ¡ A B C D → {A:B,C:D}; monadic ¡ (a b c) self-maps into a set. Membership sugar: X member Y → ⊤(X ¿ (¡ Y))

U+00A1

Glyph



Glyph — the name for a single character value. Glyph ← ≡ ⌧: Glyph's function is the literalize train (via monadic ≡) — to make a Glyph, literalize

U+00AE

Decide

138



U+00BF

Function



Function — gets glyph del (∇, nabla): the function-definition operator. Monadic ∇Y executes array Y as an FSA. (monadic 2026-10-19)

U+2207



Decide

U+00BF

Decide — first-match generalized switch/match over n×2k train tables (was When, 2026-10-11). n×2 returns 1×1 (scalar); n×4 returns 1×2; generally n×2k returns 1×k. An explicit trailing C is unmatched-result filler, not an else branch; without filler, unmatched returns Nil without logging an error. Canonical: R ← X ¿ ¡ A B C

Quote Triple



Quote Triple — « performs as Quote Single for the triple pair: parses the train into the parse buffer until » U+00BB is encountered; the U+00BB (Unquote Triple) stays for later execution (new 2026-10-21)

U+00AB

Quote Single



Quote Single — ‘ parses the train into the parse buffer until ’ U+2019 is encountered; the U+2019 (Unquote Single) stays for later execution (new 2026-10-21)

U+2018

Quote Double



Quote Double — “ performs as Quote Single for the double pair: parses the train into the parse buffer until ” U+201D is encountered; the U+201D (Unquote Double) stays for later execution (new 2026-10-21)

U+201C

(literalize)



Literalize next nonspace char, or ※0x???? by codepoint

U+2A33

Print



Print — the quad-colon verb: prints the value from its right, then passes that value to its left (new 2026-10-11)

U+2360

Rail



Rail — monadic ⌐¯X yields a vector of symbols from train X (train-to-array conversion). (new 2026-10-19)

U+2ACD

Lay



Lay — monadic ¬⊐V yields the train laid down from origin to limit of vector V (array-to-train, inverse of Rail). (new 2026-10-19)

U+2ACE

Format



Format —; monadic 𝔉X: yields the character representation of X, its display form as a character array.

U+1D571

Lambda

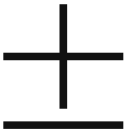


Lambda — lambda abstraction (new 2026-10-13)

U+039B

Functions: Dyadic

Collate



Collate — executes one of three trains upon a value: for less than zero, for zero, or for greater than zero (was AntiSignum ⫰ U+22B0, renamed 2026-10-12; complement to signum)

U+00B1

Curry

144



U+1D49E

Format

145



U+1D571



Curry

U+1D49E

Curry — intrinsic. Partial application: turns a dyadic verb into a monadic predicate by binding an argument, e.g. making = X from =. Semantics: Gets Q Curry P args: args to omega prime, omega prime shifts into alpha prime (binding the leftmost argument, textbook curry order), Q X makes X alpha, gets slice -1 of omega prime A, then call P; Q alpha yields P evaluated upon alpha prime, omega prime, and alpha as arguments in that order. Omega becomes nil upon Q's execution. Principle: Gets Q Curry P train is the principle for creating curries (Q is an example name assigned to the curried function, not a verb). (new 2026-10-18)



Format

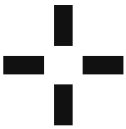
`U+1D571`

Format —; Dyadic 𝔉 S X: S is a width-and-decimals pair; each number of X is right-aligned in a field as wide as the first of S, with as many decimal digits as the second of S, rounded not truncated; too narrow a field fills with asterisks. Dyalog's ⍕ under a Wuxia glyph, provisional (new 2026-10-21)

Errors: Niladic

Silence

149



U+22B9

Compress

150



U+22D6

Errline



Last error: which line

U+274C

Errmsg



Last error: what (type)

U+274E



Silence

U+22B9

Silence — ⊹ Z V drops all nonzero values; Z transforms the zero contents in sequence. ⊹ Z M on metered M: a nonzero needle drops the pair; a zero needle selects Z (applied to the signal).; Niladic ⊹ yields boxed Nil, a trial vector for enclosing operators to work on for starts (the dual of Compress; new 2026-10-12)



Compress

U+22D6

Compress — ⋖ P V applies transform P to each nonzero content of V; zero contents are eliminated; sequence order preserved. ⋖ P M on metered M: a nonzero needle selects P (applied to the signal); a zero needle drops the pair.; Niladic ⋖ yields boxed Truth, a trial vector for enclosing operators to work on for starts (a comprehension filter; new 2026-10-12)

Cut



Cut —; niladic: on error, erases the error and returns Nil

U+22EE

Debugger



✔ ◇ *enters debugger iff error registers set*

U+2714

Errdmd



Last error: diamond position in line

U+2727

Succeed



Succeed — like Cut, except returns Truth (new 2026-10-15)

U+2999

Anticut



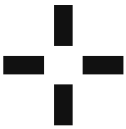
Anticut — on error throws to debugger; on return from debugger, returns Nil

U+22EF

Errors: Monadic

Silence

155



U+22B9

Compress

156



U+22D6

Cut



Cut —; monadic ⁝ X: checks X for Truth before returning

U+22EE



Silence

U+22B9

Silence — ⊹ Z V drops all nonzero values; Z transforms the zero contents in sequence. ⊹ Z M on metered M: a nonzero needle drops the pair; a zero needle selects Z (applied to the signal).; Monadic ⊹ V or ⊹ M means Z is identity: keep zeros (or zero-needle signals) as-is, drop nonzeros.



Compress

U+22D6

Compress — ⋖ P V applies transform P to each nonzero content of V; zero contents are eliminated; sequence order preserved. ⋖ P M on metered M: a nonzero needle selects P (applied to the signal); a zero needle drops the pair.; Monadic ⋖ V or ⋖ M means P is identity: pure filter, no transform.

Classes and Properties: Niladic

Constant

160

α

U+03B1

Epsilon

ε

Epsilon — holds the result of any getter, setter, or nabla when no explicit return glyph is used

U+03B5

Omega

ω

Omega — the vector of arguments passed to a setter (◅)

U+03C9

Person



Person — a callable class that emulates a person, like Muse, on the other end of a pipe: the factoring point for callbacks based in the person's responses (2026-10-20)

U+4EBA

α Constant

U+03B1

Constant — declares a class constant: X α name ← value names a constant for class X. α is also the original (pre-override) value: read-only, may not be changed directly; trying registers an error. As the original value α is an ad hoc constant — contextual to the setter, not a declared class constant

Temporary Vector



Temporary Vector — ত: holds the temporary vector of characters that Unquote Single delivers from the parse buffer; a train-local value (new 2026-10-21)

U+1D7F

Classes and Properties: Monadic

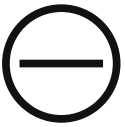
Increment



Increment —; monadic ⊕ D: invokes D's setter with nabla + 1 as the xthrement.

U+2295

Decrement



Decrement —; monadic ⊖ D: invokes D's setter with nabla - 1 as the xthrement.

U+2296

Multiply By



Multiply By —; monadic ⊛ D: invokes D's setter with nabla × 2 as the xthrement (double).

U+229B

Property



Property —; Monadic ⋘ D: yields the result of D's class call in get mode. There are no separate getters and setters: the class method gets and sets; if the method is Identity, getting and setting is transparent (2026-10-20)

U+22D8

Divide By



Divide By —; monadic ⊘ D: invokes D's setter with nabla ÷ 2 as the xthrement (halve).

U+2298

Classes and Properties: Dyadic

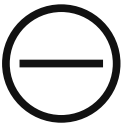
Increment



Increment —; Dyadic ⊕ D X: invokes D's setter with nabla + X as the xthrement (new 2026-10-14)

U+2295

Decrement



Decrement —; Dyadic ⊖ D X: invokes D's setter with nabla - X as the xthrement (new 2026-10-14)

U+2296

Multiply By



Multiply By —; Dyadic ⊛ D X: invokes D's setter with nabla ⨯ X as the xthrement (new 2026-10-14)

U+229B

Property



Property —; dyadic ⋘ I D: invokes D's class call in get mode, then invokes I's class call in set mode with the result as argument; yields the assigned data as a pass-through.

U+22D8

Divide By



Divide By —; Dyadic ⊘ D X: invokes D's setter with nabla ÷ X as the xthrement (new 2026-10-14)

U+2298

(method binding)



Binds method to object: X⊡function

U+22A1

Gets



Gets — ⇔A G: A stored in symbols[G], G stored in instances[A]; bidirectional binding (new 2026-10-13)

U+2B04

The Virtual: Niladic

Trapdoor



Trapdoor — the trap door to foreign code: invokes code at a given address (assembly installed via ctypes; proven 2026-10-14).

U+1D21D

VirtualSpace



VirtualSpace — the VR coordinate space: navel origin, forward half-sphere, [r;θ;φ], angles ±32767, distance in mm (new 2026-10-15)

U+1D4E5

Prim



Prim — base VR primitive: name, position, collision machinery; posted via Virtual (new 2026-10-16)

U+1D7D8

Raster



Raster — VR prim ⊑ Prim: array of pels, rank 4 [Y,X,1,3] (new 2026-10-16)

U+1D7D9

Point



Point — VR prim ⊑ Prim: dot prim, [N,6] (H,V,D,RGB) (new 2026-10-16)

U+1D7DA

Triangle



Triangle — VR prim ⊑ Prim: 9 points × 2 thetas + D + RGB = 22 numbers, [N,22] (new 2026-10-16)

U+1D7DB

The Virtual: Monadic

Trapdoor

174



U+1D21D

⟃ Trapdoor

`U+1D21D`

Trapdoor — the trap door to foreign code: invokes code at a given address (assembly installed via ctypes; proven 2026-10-14).; Monadic ⟃ A: puts the address of A into the accumulator before calling. Installer ⇔ X ⟃ ∇ body : X gets the address of the compiled body. Between the nablas different rules apply: the body is a foreign block by assembly convention, a language word and braces: asm { ... }, python { ... }, or c { ... }; inside the braces the named language's rules govern, not Wuxia's (2026-10-21). If X is so bound, niladic X calls the code. ε maps to the accumulator. The call yields ε

Dungeon

Dungeon: Places

Where

@

Where the hero is — bound to '@', the roguelike player glyph. Where⊡Zee yields Where [-1 (last); Where⊡Wye yields Where [1 (midposition — middle of 3 coords); Where⊡Exx yields Where [0 (zero). INDICES START AT ZERO

U+0040

Dungeon

£

The Dungeon — literally the set of all levels

U+00A3

Screen

§

The Screen — intrinsic 20×60 array (16 play + 4 I/O rows)

U+00A7

Level

¶

Level — the pilcrow: a single dungeon level, member of £ Dungeon

U+00B6

Cell

¢

A dungeon cell — the cent sign joins the family: § £ ¢. Cell⊡location ← £ Where: the cell's location is the Dungeon at Where. A dungeon square IS a dictionary of instances (the ⓘ registry's entries present at that square)

U+00A2

Up



Up — the upward/north movement. Where⊡up yields ∅ when Where⊡Zee is 0, else Where with Zee decremented (refined 2026-10-10: boundary-checked — top edge is a quiet Nil, not a negative index)

U+25B2

Down



Down — the downward/south movement, counterpart to Up. Where⊡down yields ∅ when Where⊡Zee ≥ Dungeon⊡Zee (bottom edge), else Where⊡Zee + 1 (confirmed 2026-10-10)

U+25BC

Dungeon: Classes

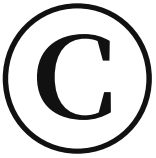
Class



Class — the circled dot. X⊡Class gets a RANK-2 ARRAY OF TRAINS: a class IS a decision table made of trains. Class instances are callable; the train defaults to Nil (∅)

U+2299

Classes



Classes — the set of all classes (plural of Class), parallel to £ Dungeon as the set of all levels. Implementation: a single twin (bidirectional word↔glyph) dictionary at global scope

U+24B8

Instances

182



U+24BE

Instance



Instance — the INDIVIDUAL instance (singular), distinct from Ⓘ the table. Joins the blackboard family (𝕏 𝕐 ℤ ℙ 𝕆). Carries the (glyph, hash-at-creation) identity tuple

U+1D540



Instances

U+24BE

Instances — the TABLE/set of all instances (plural), parallel to Ⓒ Classes. Implementation: a single twin (bidirectional word ↔ glyph) dictionary at global scope. Instance identity is the (glyph, hash-at-creation) tuple, used as the key field in both the global instance dictionary and each square (Cell) dictionary

Interpreter



Interpreter — the Wuxia interpreter (new 2026-10-15)

U+203D

Inherits



Inherits — the inheritance operator (square subset of)

U+2291

Isa



Isa — the is-a / instantiation operator (2026-10-10). Monadic ∈ Y: Y is a class to copy data from; new instance.ε = Y.ε. Dyadic ∈ Y X: X is a class, optional initializer; new instance.ε = X.ε (new 2026-10-14)

U+22FF

Object



Object — a DUNGEON object, not a generic OOP object. The blackboard O joins the projection family (𝕏 𝕐 ℤ ℙ): the root of the dungeon's object hierarchy, what every dungeon entity inherits from

U+1D546

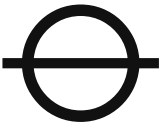
VirtualDistance



VirtualDistance — distance in VirtualSpace, 0-512 (new 2026-10-15)

U+27F0

Angle



Angle — a VR angle (theta/phi) (new 2026-10-15)

U+29B5

VirtualHorizontal



VirtualHorizontal — horizontal angle in VirtualSpace (new 2026-10-15)

U+29D3

VirtualVertical



VirtualVertical — vertical angle in VirtualSpace (new 2026-10-15)

U+29D7

Dungeon: Objects

Ring



Ring — the blackboard R: a dungeon object (ring), third of the dungeon entity glyphs

U+211D

Wand



Wand — a dungeon object (wand)

U+2A21

Armor



Armor — the blackboard A: a dungeon object (armor), first of the dungeon entity glyphs

U+1D538

Weapon



Weapon — the blackboard W: a dungeon object (weapon), second of the dungeon entity glyphs

U+1D54E

Amulet



Amulet — eight-petalled floret: a dungeon object (amulet). (Was U+1DF76, tofu on device — swapped 2026-10-10)

U+2741

Food



Food — the blackboard F: a dungeon object (food)

U+1D53D

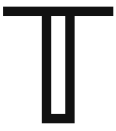
Scroll



Scroll — the blackboard S: a dungeon object (scroll)

U+1D54A

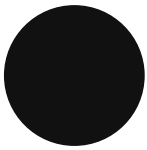
Tool



Tool — the blackboard T: a dungeon object (tool)

U+1D54B

Gem



Gem — solid black circle: a dungeon object (gem). (Was U+2733 — swapped 2026-10-10)

U+25CF

Potion



Potion — heavy exclamation: a dungeon object (potion)

U+2757

Dungeon: Entities

Player



Player — black smiling face: the dungeon's player entity. Inherits Monster plus a few extra fields Monster does not have

U+263B

Monster



Monster — the blackboard M: a dungeon monster, first of the monster glyphs

U+1D544

Dungeon: Food

Tin



Tin — circled white star: a dungeon food item (tin), first of the object species

U+272A

RefriedBeans



RefriedBeans — dark-encircled six: grants teleportation

U+277B

Spinach



Spinach — dark-encircled seven: Popeye strength

U+277C

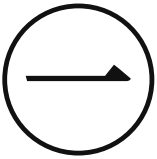
Sushi



Sushi — circled congratulation: the sushi/sashimi food category. (Was U+2722 — swapped 2026-10-11)

U+3297

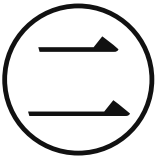
CarpSashimi



CarpSashimi — circled kanji: sushi item

U+3280

WhaleSashimi



WhaleSashimi — circled kanji: sushi item

U+3281

EelSashimi



EelSashimi — circled kanji: sushi item

U+3282

SalmonSashimi



SalmonSashimi — circled kanji: sushi item

U+3283

TunaSashimi



TunaSashimi — circled kanji: sushi item

U+3284

CrabSashimi



CrabSashimi — circled kanji: sushi item

U+3285

DragonSashimi



DragonSashimi — circled kanji: sushi item

U+3286

TurtleSashimi



TurtleSashimi — circled kanji: sushi item

U+3287

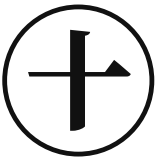
Oyster



Oyster — circled kanji: sushi item

U+3288

Uni



Uni — circled kanji: sushi item

U+3289

Gyuniku



Gyuniku — circled kanji: sushi item

U+328A

Asian



Asian — squared hand: the Asian food category. (Was U+328C — swapped 2026-10-11)

U+1F210

Ramen



Ramen — squared cjk unified ideograph-5b57: Asian food

U+1F211

FortuneCookie



FortuneCookie — squared cjk unified ideograph-53cc: Asian food

U+1F212

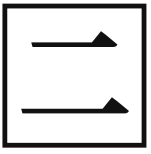
Kelp



Kelp — squared katakana de: Asian food

U+1F213

Dumplings



Dumplings — squared cjk unified ideograph-4e8c: Asian food

U+1F214

BowlOfRice



BowlOfRice — squared cjk unified ideograph-591a: Asian food

U+1F215

SpringRolls



SpringRolls — squared cjk unified ideograph-89e3: Asian food

U+1F216

Tofu



Tofu — squared cjk unified ideograph-5929: Asian food

U+1F217

PreservedEgg



PreservedEgg — squared cjk unified ideograph-4ea4: Asian food

U+1F218

Soup

⑳

Soup — circled twenty: the soup category

U+2473

Deli

❖

Deli — black diamond minus white X: the delicatessen food category (the 25 staples, names lost 2026-10-10 — rebuild pending)

U+2756

Fruit

🁰

Fruit — domino tile back: the fruit food category

U+1F030

CantoneseDuck



CantoneseDuck — squared cjk unified ideograph-6620: Asian food

U+1F219

Egg

Ø

Egg — o with stroke: the egg. (Was U+2800 — swapped 2026-10-11)

U+00F8

Meat



Meat — CJK beautiful: the meat category

U+4E3D

Creature



Creature — o with long stroke overlay: the monster-shuffle category

U+A74B

Candy



Candy — mahjong joker: the candy category

U+1F02A

Goodbar



Goodbar — mahjong east wind: candy

`U+1F000`

Honee



Honee — mahjong south wind: candy

`U+1F001`

DRation



DRation — mahjong west wind: candy

`U+1F002`

DoveBar



DoveBar — mahjong north wind: candy

`U+1F003`

AcidBroth

❶

AcidBroth — dark-encircled number: soup

U+2776

GoldenSoup

❷

GoldenSoup — dark-encircled number: soup

U+2777

StonedSoup

❸

StonedSoup — dark-encircled number: soup

U+2778

CrowBar



CrowBar — mahjong winter: candy

U+1F029



MoldySoup

MoldySoup — dark-encircled number: soup

U+2779



Manwich

Manwich — dark-encircled number: soup

U+277A



CreamOfAlgaeSoup

CreamOfAlgaeSoup — dark-encircled number: soup

U+277E



ChickenNoodleSoup

ChickenNoodleSoup — dark-encircled number: soup

U+277F

Apple



Apple — domino tile: fruit, inherits Fruit

U+1F032

Orange



Orange — domino tile: fruit, inherits Fruit

U+1F033

Pear



Pear — domino tile: fruit, inherits Fruit

U+1F034

Melon



Melon — domino tile: fruit, inherits Fruit

U+1F035

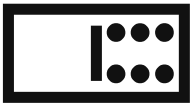
Banana



Banana — domino tile: fruit, inherits Fruit

U+1F036

Carrot



Carrot — domino tile: fruit, inherits Fruit

U+1F037

Fig



Fig — domino tile: fruit, inherits Fruit

U+1F038

Lemon



Lemon — domino tile: fruit, inherits Fruit

U+1F039

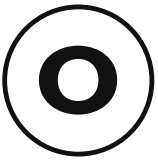
Tomb



Tomb — circled 10 on black square: safe-row tin, now Sundry

U+3248

Sundry



Sundry — circled ideograph: the sundry/miscellaneous category

U+3267

Lime



Lime — domino tile: fruit, inherits Fruit

U+1F03A

Grapes



Grapes — domino tile: fruit, inherits Fruit

U+1F03B

Pesto



Pesto — circled 20 on black square: safe-row tin, now Sundry

U+3249

KelpFrond



KelpFrond — circled water: sundry item (kelp is seaweed)

U+328C

EucalyptusLeaf



EucalyptusLeaf — circled wood: sundry item

U+328D

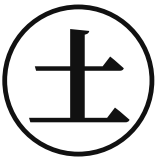
CandyBar



CandyBar — circled metal: sundry item

U+328E

LembasWafer



LembasWafer — circled earth: sundry item

U+328F

CramRation



CramRation — circled sun: sundry item

U+3290

FoodRation



FoodRation — circled medicine: sundry item

U+32A9

KRation



KRation — circled religion: sundry item

U+32AA

CloveOfGarlic



CloveOfGarlic — circled attention: sundry item

U+329F

SprigOfWolfsbane



SprigOfWolfsbane — circled item: sundry item

U+32A0

LumpOfRoyalJelly



LumpOfRoyalJelly — circled copy: sundry item

U+32A2

CRation



CRation — circled study: sundry item

U+32AB

Dungeon: Species inheritance

EarthElemental



⊑ Elementals — Species inherits its category (2026-10-10)

U+2295

FireElemental



⊑ Elementals — Species inherits its category (2026-10-10)

U+229B

AirElemental



⊑ Elementals — Species inherits its category (2026-10-10)

U+2A37

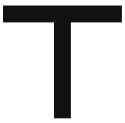
WaterElemental



⊑ Elementals — Species inherits its category (2026-10-10)

U+2A38

PurpleWorm



⊑ Worms — Species inherits its category (2026-10-10)

U+22A4

BabyPurpleWorm



⊑ Worms — Species inherits its category (2026-10-10)

U+22A6

LongWorm



⊑ Worms — Species inherits its category (2026-10-10)

U+2A53

BabyLongWorm



⊑ Worms — Species inherits its category (2026-10-10)

U+2A55

LargeKobold



⊑ Kobolds — Species inherits its category (2026-10-10)

U+049C

Kobold



⊑ Kobolds — Species inherits its category (2026-10-10)

U+049D

MasterMindFlayer



⊑ Mindflayers — Species inherits its category (2026-10-10)

U+04F4

MindFlayer



⊑ Mindflayers — Species inherits its category (2026-10-10)

U+04F5

StoneGiant

G˙

⊑ Giants — Species inherits its category (2026-10-10)

U+0120

Giant

Ģ

⊑ Giants — Species inherits its category (2026-10-10)

U+0122

KoboldLord

Ќ

⊑ Kobolds — Species inherits its category (2026-10-10)

U+040C

KoboldShaman

Ҝ

⊑ Kobolds — Species inherits its category (2026-10-10)

U+049B

Ĝ

FireGiant

⊑ *Giants — Species inherits its category (2026-10-10)*

`U+011C`

Ğ

HillGiant

⊑ *Giants — Species inherits its category (2026-10-10)*

`U+011E`

Ǵ

FrostGiant

⊑ *Giants — Species inherits its category (2026-10-10)*

`U+0193`

G ˇ

Ettin

⊑ *Giants — Species inherits its category (2026-10-10)*

`U+01E6`

VampireBat



⊑ Vampires — Species inherits its category (2026-10-10)

U+0180

StormGiant



⊑ Giants — Species inherits its category (2026-10-10)

U+01E4

Vampire



⊑ Vampires — Species inherits its category (2026-10-10)

U+1E7F

Titan



⊑ Giants — Species inherits its category (2026-10-10)

U+2141

HumanMummy



⊑ Undead — Species inherits its category (2026-10-10)

U+046A

ElfMummy



⊑ Undead — Species inherits its category (2026-10-10)

U+046A

VampireLeader



⊑ Vampires — Species inherits its category (2026-10-10)

U+1E7D

Nosferatu



⊑ Vampires — Species inherits its category (2026-10-10). In this mod, Vlad becomes Nosferatu: Vlad is deleted and his versicle Ṽ U+2123 is Nosferatu's (2026-10-20)

U+2123

DwarfMummy



⊑ *Undead — Species inherits its category (2026-10-10)*

U+046A

GnomeMummy



⊑ *Undead — Species inherits its category (2026-10-10)*

U+046A

OrcMummy



⊑ *Undead — Species inherits its category (2026-10-10)*

U+046A

KoboldMummy



⊑ *Undead — Species inherits its category (2026-10-10)*

U+046A

GiantMummy



⊑ Undead — Species inherits its category (2026-10-10)

U+046A

EttinMummy



⊑ Undead — Species inherits its category (2026-10-10)

U+046A

KoboldZombie



⊑ Undead — Species inherits its category (2026-10-10)

U+2124

GnomeZombie



⊑ Undead — Species inherits its category (2026-10-10)

U+2124

OrcZombie



⊑ Undead — Species inherits its category (2026-10-10)

U+2124

DwarfZombie



⊑ Undead — Species inherits its category (2026-10-10)

U+2124

ElfZombie



⊑ Undead — Species inherits its category (2026-10-10)

U+2124

HumanZombie



⊑ Undead — Species inherits its category (2026-10-10)

U+2124

Ghoul

𝑔

⊑ Undead — Species inherits its category (2026-10-10)

U+210A

GiantZombie

ℤ

⊑ Undead — Species inherits its category (2026-10-10)

U+2124

EttinZombie

ℤ

⊑ Undead — Species inherits its category (2026-10-10)

U+2124

WoodElf



⊑ Elves — Species inherits its category (2026-10-10)

U+2618

GreenElf



⊑ Elves — Species inherits its category (2026-10-10)

U+2618

GreyElf



⊑ Elves — Species inherits its category (2026-10-10)

U+2618

Elf



⊑ Elves — Species inherits its category (2026-10-10)

U+2618

ElfNoble



⊑ Elves — Species inherits its category (2026-10-10)

U+2E19

Centipede

¢

⊑ Spiders — Species inherits its category (2026-10-10)

U+00A2

Dragon

Δ

⊑ Dragons — Species inherits its category (2026-10-10)

U+0394

BabyDragon

δ

⊑ Dragons — Species inherits its category (2026-10-10)

U+03B4

ElvenMonarch



⊑ Elves — Species inherits its category (2026-10-10)

U+2F40

CaveSpider

¤

⊑ Spiders — Species inherits its category (2026-10-10)

U+00A4

Scorpion

±

⊑ Spiders — Species inherits its category (2026-10-10)

U+00B1

GiantSpider

Ø

⊑ Spiders — Species inherits its category (2026-10-10)

U+00D8

Mastodon

Ψ

⊑ Beasts — Species inherits its category (2026-10-10)

U+03A8

Mumak

Ψ

⊑ Beasts — Species inherits its category (2026-10-10)

U+03A8

Baluchitherium

ϒ

⊑ Beasts — Species inherits its category (2026-10-10)

U+03D2

Minotaur

Ṁ

⊑ Beasts — Species inherits its category (2026-10-10)

U+1E40

Titanothere

ℴ

⊑ Beasts — Species inherits its category (2026-10-10)

U+213A

Wumpus

¿

⊑ Beasts — Species inherits its category (2026-10-10)

U+00BF

Bugbear

ö

⊑ Beasts — Species inherits its category (2026-10-10)

U+00F6

Owlbear

ø

⊑ Beasts — Species inherits its category (2026-10-10)

U+00F8

Trapper

Й̨

⊑ Beasts — Species inherits its category (2026-10-10)

U+048A

Jabberwock

ƸƷ

⊑ Beasts — Species inherits its category (2026-10-10)

U+01B8

Rothe

Ȣ

⊑ Beasts — Species inherits its category (2026-10-10)

U+0222

Leocrotta

Ł

⊑ Beasts — Species inherits its category (2026-10-10)

U+023D

LurkerAbove



⊑ Lurkers — Species inherits its category (2026-10-10)

U+214F

Shade

@

⊑ Guises — Species inherits its category (2026-10-10)

U+0040

Skeleton

@

⊑ Guises — Species inherits its category (2026-10-10)

U+0040

Foocubus

@

⊑ Guises — Species inherits its category (2026-10-10)

U+0040

Stalker

Ǝ

⊑ Lurkers — Species inherits its category (2026-10-10)

U+042D

MountainNymph



⊑ *Guises — Species inherits its category (2026-10-10)*

U+0040

WoodNymph



⊑ *Guises — Species inherits its category (2026-10-10)*

U+0040

WaterNymph



⊑ *Guises — Species inherits its category (2026-10-10)*

U+0040

MerfolkRoyal



⊑ *Merfolk — Species inherits its category (2026-10-10)*

U+03D0

Shopkeeper

§

⊑ Specials — Species inherits its category (2026-10-10)

U+00A7

MerfolkWitch

o´

⊑ Merfolk — Species inherits its category (2026-10-10)

U+03CC

MerfolkCommon

ȯ

⊑ Merfolk — Species inherits its category (2026-10-10)

U+1F40

MerfolkNoble

ȫ

⊑ Merfolk — Species inherits its category (2026-10-10)

U+1F42

WizardOfYendor

¥

⊑ Specials — Species inherits its category (2026-10-10)

U+00A5

Tin



⊑ Food — Tin inherits Food — first object species joins the hierarchy (2026-10-10)

U+272A

AcidBroth



⊑ Soup — Soup inherits Soup (2026-10-10)

U+2776

GrandMaster

父

⊑ Specials — Species inherits its category (2026-10-10)

U+7236



GoldenSoup

⊏ Soup — Soup inherits Soup (2026-10-10)

U+2777



StonedSoup

⊏ Soup — Soup inherits Soup (2026-10-10)

U+2778



MoldySoup

⊏ Soup — Soup inherits Soup (2026-10-10)

U+2779



Manwich

⊏ Soup — Soup inherits Soup — the human-flesh tin (2026-10-10)

U+277A



RefriedBeans

⊑ Soup — Soup item inherits Soup — grants teleportation (2026-10-11)

U+277B



CreamOfAlgaeSoup

⊑ Soup — Soup inherits Soup (2026-10-10)

U+277E



ChickenNoodleSoup

⊑ Soup — Soup inherits Soup (2026-10-10)

U+277F

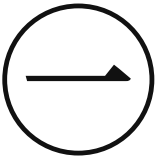


Sushi

⊑ Food — Sushi inherits Food — the sushi food category (2026-10-10)

U+3297

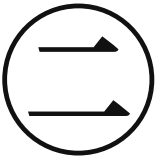
CarpSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3280

WhaleSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3281

EelSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3282

SalmonSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3283

TunaSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3284

CrabSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3285

DragonSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3286

TurtleSashimi



⿸ Sushi — Individual sushi
inherits Sushi (2026-10-10)

U+3287

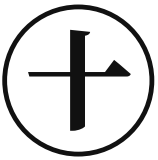
Oyster



⊑ Sushi — Individual sushi inherits Sushi (2026-10-10)

U+3288

Uni



⊑ Sushi — Individual sushi inherits Sushi (2026-10-10)

U+3289

Gyuniku



⊑ Sushi — Individual sushi inherits Sushi (2026-10-10)

U+328A

Asian



⊑ Food — Asian inherits Food — the Asian food category (2026-10-10)

U+1F210

Ramen



⿺ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F211

FortuneCookie



⿺ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F212

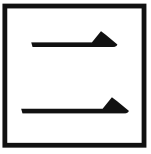
Kelp



⿺ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F213

Dumplings



⿺ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F214

BowlOfRice



⿸ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F215

SpringRolls



⿸ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F216

Tofu



⿸ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F217

PreservedEgg



⿸ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F218

Deli



⊑ Food — Deli inherits Food — the delicatessen food category (2026-10-10)

U+2756

KelpFrond



⊑ Sundry — Sundry item inherits Sundry (2026-10-10)

U+328C

EucalyptusLeaf



⊑ Sundry — Sundry item inherits Sundry (2026-10-10)

U+328D

CantoneseDuck



⊑ Asian — Individual Asian food inherits Asian (2026-10-10)

U+1F219

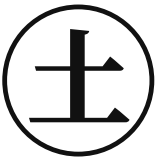
CandyBar



⿺ Sundry — Sundry item inherits Sundry (2026-10-10)

U+328E

LembasWafer



⿺ Sundry — Sundry item inherits Sundry (2026-10-10)

U+328F

CramRation



⿺ Sundry — Sundry item inherits Sundry (2026-10-10)

U+3290

FoodRation



⿺ Sundry — Sundry item inherits Sundry (2026-10-10)

U+32A9

CloveOfGarlic



⿸ Sundry — Sundry item inherits Sundry (2026-10-10)

U+329F

SprigOfWolfsbane



⿸ Sundry — Sundry item inherits Sundry (2026-10-10)

U+32A0

KRation



⿸ Sundry — Sundry item inherits Sundry (2026-10-10)

U+32AA

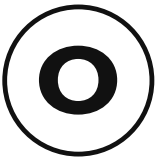
CRation



⿸ Sundry — Sundry item inherits Sundry (2026-10-10)

U+32AB

Sundry



⊏ Deli — Sundry is part of Deli (2026-10-10)

U+3267

LumpOfRoyalJelly



⊏ Sundry — Sundry item inherits Sundry (2026-10-10)

U+32A2

Fruit



⊏ Food — Fruit inherits Food — the fruit food category (2026-10-10)

U+1F030

Apple



⊏ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F032

Orange



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F033

Pear



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F034

Melon



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F035

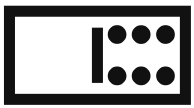
Banana



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F036

Carrot



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F037

Fig



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F038

Lemon



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F039

Lime



⊑ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F03A

Soup



⿺ Tin — Soup inherits Tin — the soup category (2026-10-11)

U+2473

Goodbar



⿺ Candy — Goodbar inherits Candy (2026-10-11)

U+1F000

Candy



⿺ Food — Candy inherits Food — the candy category (2026-10-11)

U+1F02A

Grapes



⿺ Fruit — Fruit item inherits Fruit (2026-10-10)

U+1F03B

Honee



⊑ Candy — Honee inherits Candy
(2026-10-11)

U+1F001

DRation



⊑ Candy — DRation inherits Candy
(2026-10-11)

U+1F002

DoveBar



⊑ Candy — DoveBar inherits Candy
(2026-10-11)

U+1F003

CrowBar



⊑ Candy — CrowBar inherits Candy
(2026-10-11)

U+1F029

Spinach



⊑ Soup — Spinach (NetHack tin filling, Popeye strength) inherits Soup (2026-10-11)

U+277C

Tomb



⊑ Sundry — Tomb inherits Sundry (2026-10-11)

U+3248

Pesto



⊑ Sundry — Pesto inherits Sundry (2026-10-11)

U+3249

Meat



⊑ Creature — Meat inherits Creature — the meat category (2026-10-11)

U+4E3D

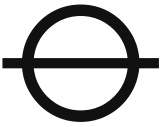
Egg



⊏ Creature — Egg inherits Creature (2026-10-11)

U+00F8

Angle



⊑ Integer — Angle inherits Integer (IntegerBounded removed 2026-10-20) (new 2026-10-15)

U+29B5

VirtualHorizontal



⊑ Angle — VirtualHorizontal inherits Angle (new 2026-10-15)

U+29D3

Creature



⊏ Tin — Creature inherits Tin — the creature-meat tin category; the category that shuffles monsters as consumables (tins of monster meat) (2026-10-11)

U+A74B

Egg

Ø

Egg stays empty till use — populated by the Creature monster-shuffle at runtime (2026-10-11)

U+00F8

VirtualDistance



⊑ Integer — VirtualDistance inherits Integer (IntegerBounded removed 2026-10-20) (new 2026-10-15)

U+27F0

VirtualVertical



⊑ Angle — VirtualVertical inherits Angle (new 2026-10-15)

U+29D7

Meat



Meat stays empty till use — populated by the Creature monster-shuffle at runtime (2026-10-11)

U+4E3D

Appendix: Sugars and Exchanges

Wuxia is its own language, but nobody arrives at it from nowhere. Most readers come from another array language: Dyalog APL, J, classic APL. This appendix lists the concessions made to those readers, so a familiar operation can be found without hunting.

Three kinds of concession appear here. A sugar is a familiar operation spelled from Wuxia parts, given one glyph or one idiom so it reads as a unit. An exchange is a glyph trade: the other language's glyph could not come over, because Wuxia already spends it on something else, so the meaning arrives under a Wuxia glyph instead. An import is the opposite case: the other language's glyph was free, and it came over as it stands.

Sugars

- Divide Matrix ⌹ X Y is sugar for ⍤ ¯1 X ÷Y: Inner Product of X with the inverse of Y. Monadic ÷ on a square array yields the inverse, so the composition came first (settled 2026-10-14) and the glyph followed (2026-10-21).
- Link ⧺ is dyadic sugar for Catenate Map Box.
- Raze ⫴ is monadic sugar for Ravel Map Unbox.
- Roots ℞ is sugar over Reveal ℜ and NextLocal ℓ.
- The die faces D2 ⚀, D4 ⚁, D6 ⚂, D10 ⑩, D12 ⑫ are sugar for ⅅ n, Dice of n.
- Reverse First, Dyalog's ⊖, is the transpose sandwich ⍉ ⌽ ⍉. Wuxia keeps one Reverse and pays for the pair with a composition: Dyalog's two glyphs are sugar, not power, against Wuxia's one (as of v1.1).
- And and Or are read through the quantifiers: And is ∀

𝕄 X Y, Or is ∃ 𝕄 X Y. Not Match is ¬ ≙ A B.

- Pi Times, Dyalog's monadic ○, is spelled × π. It is manual in this language, on purpose.
- Table, Dyalog's ⍪, is Extend 𝔼 by definition: monadic Extend adds a trailing axis of length 1.
- From J: Roots is the refactoring above, and the nub sieve is an idiom, ⍢ ⊚ 𝒞 = A ⋄ train, first entry in the idioms.

Exchanges

- Encode and Decode. Dyalog writes them ⊤ and ⊥; here ⊤ is Truth, and Truth does not move. They arrive as ⫟ U+2AE7 and ⫠ U+2AE8, the short tacks with bars, kin to the faces they replace, with Dyalog's semantics whole.
- Format. Dyalog's ⍕ stays in Dyalog. Wuxia gives Format 𝔉 U+1D571: bold fraktur, because formatting is an entire subsystem (field widths, decimals, rounding, alignment), and bold fraktur capitals are the subsystem shelf. Provisional.
- Expand. Dyalog's \ is Scan in Wuxia, the backstroke, and it does not do double duty here. Expand arrives as 𝔈 U+1D570, with Stop ⋄ separating the pattern from the array.
- Mix and Split. Dyalog's ↑ and ↓ are Take and Drop here, moving elements. Mix and Split move a whole nesting level, so they take the doubled arrows: Mix ⇑ U+21D1, Split ⇓ U+21D3. Single arrows move elements; double arrows move a level.
- Depth. Dyalog's ≡ is Congruent in Wuxia, the same-binding test, and it does not move either. Depth

takes ⍤ U+2355, with Dyalog's definition: a scalar has depth 0, an array of scalars depth 1, each further layer of nesting or boxing adds one, and the deepest chain governs.

- Match. Dyalog's ≢ is tally here. Match takes ≗ U+2257: Truth when two arrays agree in shape, rank, depth, values, and types throughout, the verdict a single Truth or Nil. Wuxia's = is the looser elementwise Equality; Match is the strict one.
- Index. Dyalog's [] stays in Dyalog. The work is done by Glean ⋈ U+27D7, under a Wuxia name: one index set per axis, their cross product, as in Dyalog.
- Partitioned Enclose. Dyalog's ⊂ is Box here, twice over, and does not move. Dyalog itself later gave Partition its own glyph, and Wuxia takes that one: Partition ⊆ U+2286, dyadic as specified.

Imports, as they stand

- Left ⊣ U+22A3 and Right ⊢ U+22A2, as specified in Dyalog: monadic Same, dyadic argument selection. Room was made for Left deliberately: the dungeon's Wand moved from ⊣ to ↾ U+21BE first.
- Find ⍷ U+2377, as specified in Dyalog: a Boolean array the shape of the haystack, Truth at the start of each occurrence of the needle.
- Dyadic Transpose ⍉, as specified in Dyalog: the permutation form and the diagonal form both. Some things are not to be redesigned, because users already know them.
- Residue needed no import: dyadic % was already

remainder.

Not conceded

- Execute. Dyalog's ⍎ runs text as code, direct. That form stays refused. But execution itself is implemented, via Nabla: raw text is assembled into a train, a vector, or an array first, and ∇ executes the assembled artifact. The quoted-passage idiom ∇ '...' ⍨ is the way in.
- Least squares. Divide Matrix covers square systems. The non-square least-squares case stays out: no pseudoinverse has been added, by design.
- J's i: (integers in both directions) was declined without a glyph.

Still open

Nothing is left open. Partitioned Enclose, the last item on the board, arrived as Partition ⊆ (see Exchanges). The review that produced this appendix is complete.

Appendix: Back Rank

A monad is learned once, on a scalar. This appendix states the rule that lets you use it everywhere else:

All monads can operate on back-rank scalars.

The back rank of an array is its last rank: the scalars the array is finally made of. A monadic verb applied to an array does not need a new definition for the array. It walks to the back rank and applies itself to each scalar there, and the result keeps the array's shape. The verb you learned on one number is the verb you get on a million.

Examples

Negate, on a vector:

```
- (3 ¯5 2) yields (¯3 5 ¯2)
```

Each scalar changes sign. Nothing else about the array moves.

Floor, on a matrix:

```
⌊ (1.7 2.3 ¯0.4) yields (1 2 ¯1)
```

Monadic Floor rounds each scalar down, in place, whatever the shape above it.

Signum, on a vector:

```
+ (7 ¯2 0) yields (1 ¯1 0)
```

Monadic Plus is Signum, and Signum is a scalar question. The back-rank rule is what lets it answer for a whole array at once.

Invert, on truth values:

```
~ (T Nil T) yields (Nil T Nil)
```

Each back-rank scalar is inverted; the vector's shape and order stand.

Where the whole array wins

Some monads also carry a whole-array meaning, and where that meaning applies, it governs. Monadic Quotient on a square array is the matrix inverse, not a million reciprocals:

```
÷ (square array) yields its inverse
÷ (2 4 5) yields (0.5 0.25 0.2)
```

Off the square, Quotient is a back-rank scalar operator, the reciprocal. The dictionary entries for Quotient and Star said "back-rank scalar operator" from early on; this rule is those words grown to cover every monad.

Dictionaries

The back rank of a dictionary is its values. A monad applied to a dictionary applies to each value at the back rank, and the keys stand untouched:

```
- (scores: 3, fouls: ¯1) yields (scores: ¯3, fouls: 1)
```

Why the rule exists

Without it, every monad would need two definitions, one for scalars and one for arrays, and the reader would need both. With it there is one definition, stated where the verb is defined, and the back-rank rule carries it to any shape. It is the same reason sugar exists: a concept made almost mindless to remember and use, because the walk to the scalars is never the reader's problem.

Appendix: Persons

Persons began as plumbing. A game needs to talk to whoever is on the other end of the wire: the player at the console, an NPC with a role to play, a teammate, a professional answering in their hat. They ended as something larger: the thing generators are made of. This appendix tells both halves.

The interface

A Person 人 U+4EBA is a callable class: a person emulated at the far end of a pipe. At the bottom the pipe is two-way binary, full duplex, bytes. The Person is the framing layer over those bytes: where one message ends, which reply belongs to which question, what silence means, and room for speech nobody asked for, because people talk out of turn.

Person is a role, not a species. A monster on the board is a decision table wearing a glyph. An NPC is a Person invited to play: given a seat, a pipe, a role, and a body in the dungeon. Without the invitation it is just a monster piece running its table. The player is a Person too, and so is a teammate or a professional once connected as a live respondent: the profession is the hat worn, the Person is the answering presence behind the pipe.

The Hero is the Person whose pipe ends at the local console, and so gains the keyboard and the screen. To open a connection, instance a Person with initialize(s): that establishes a one-way or two-way connection. A one-way Person forks into a source, which only speaks, and a sink, which only listens. At startup both ends are stamped: Me.input gets identity, and Me.output gets identity.

Event trains

A Person runs on an event train. The default mathematical event behind a Person at startup is Time ⌚, and a Person can always be set running on zeroes or on ones instead: no Person starts blocked. A Person does not inherit Time. A person is in time, not a kind of time: the Person has a timeline, and the event is plugged in through initialize. Change the event train and the same Person serves differently. Instance a Person and plug Time in as the math event and you have WWV, the time station: a Person whose whole soul is the clock. Swap the event train for the keyboard and you have a player. Service Persons run on Clock ⏱️, the blocking heartbeat: block a tenth-count, wake, serve, block again.

Generators

A generator is an indefinite vector: indefinite in length and indefinite in content, because each element does not exist until its coordinate is read. Wuxia has no generator primitive, and needs none:

People form the basis of generators.

Anything that would be a generator is a Person standing on an event train, carrying its own state. No outside nabla is needed: the state the series is formed by lives inside the Person, events knock, and the state answers. No yield is needed either, ever: there is no straight-through execution to suspend. The three refusals stand together, no loops, no interrupts, no yield.

Fibonacci, as a Person: Leonardo on the pipe carries the

pair (a, b) in himself. Each event's train yields the elder to ε and leaves the pair shifted and added. The first mathematical event is simpler still: an unlimited series of zeroes, a source that never yields Nil, so the stopping lives in the asker.

Reading from a generator spends it or not, by what backs it. An inner product between a fixed vector and a generator lends the generator its length: that many elements are read, and the contraction closes. Against a stateless, computed source the reads recompute and the product is repeatable. Against a Person the reads are consumed: the product is stamped by where in the Person's life it was taken, and asking again buys a different answer.

Cautions

A Person backed by Time is a random number source, and the samples stay pseudorandom because sampling is incommensurate with the turning wheel. That is an application note, and the reason not to back Dice with one: the source is public and deterministic, so anyone holding the stamp can reproduce the samples, and clock-driven rolls are gameable wherever fairness matters. Trust Time for the bit, never for the number.

Blocking input makes a lousy generator for the same family of reason: wake times pollute the stream, and the samples end up measuring the waiter's rhythm instead of the wheel. Blocking is for commo: keystrokes, pipes, Persons talking. Generators sample on their own event trains, never on a blocker's wake.

Appendix: Evaluation Order

Glyphs ask for arguments of a kind: this one wants a scalar, that one a vector, another an array of some rank. What arrives does not always match what was asked. This appendix states the order in which a mismatch is resolved, and why the order runs the way it does.

Where an array is expected, the argument is read in this order:

1. Array. If it already is an array, it is used as it stands, untouched. 2. Train. Else, if it is a train, it is read as data: its symbols taken as Rail takes them, never executed. 3. Scalar. Else, if it is a scalar, it is read as [scalar]: a vector of one element, the scalar itself.

Which slot a glyph offers, and so which reading applies, is recognized by the nature of the glyph executing. ∇ executes what it is given; a glyph that takes data reads what it is given.

The array stands

The first rule is that no rule is needed. An array that arrives at an array slot is already what was asked for, so it passes through with no reading applied at all.

This has to come first, because the other two readings are conversions, and a conversion applied to genuine data corrupts it. A train of length L and a one-row array of the same symbols carry the same shapes (ρ of either is 1 L), so if the train reading ever outranked the array reading, a real one-row array could be silently re-read as a train and no array of symbols would be safe in its own slot. Array first means conversions happen only when the direct reading is impossible.

The train reads

A train is not an array, but it is data-shaped: a row of symbols. The train/vector rule admits it to a vector slot, with one guard: it is read, never run. Execution is ∇'s office alone, so a train handed to a data glyph loses nothing and fires nothing. There is one gate, ∇, and this is it: reading and assembly execute nothing, ever; running happens under the executing glyph alone.

The scalar wraps

A scalar in an array slot is the weakest claim, so it receives the strongest reading: it becomes a vector of length one containing itself.

```
a slot expecting a vector, given 5, holds (5): ρ yields 1
```

The wrap is [scalar], not broadcast. Broadcasting would fabricate elements the user never supplied, padding the scalar out to whatever shape the slot wanted and pretending the array was always that size. The one-element vector tells the truth about what happened: one scalar arrived, and here it is, in an array. If a larger shape is genuinely wanted, that is an act, and the language has the verb for it: dyadic ρ reshapes, explicitly, where everyone can see it.

Why this order

Least transformation wins. Each reading down the list interprets its argument more than the one above it: the array is interpreted not at all, the train is reinterpreted in kind, the scalar is re-housed entirely. Trying them in that order means a value is always taken at

its most-formed reading, and a stronger reading is never displaced by a weaker one that happens also to fit. The payoff is trust. A reader of a train can know that what was data stays data, that nothing they wrote as an array will come back re-read as code or as a train, and that the one conversion they cannot see coming, the scalar wrap, is the smallest one that could possibly be made.

Appendix: Extending Wuxia

This language is finished, and it is not finished, and both are deliberate. The author fenced off what he had not settled and left the fence with a gate in it: trapdoors and open glyph assignment, so that someone else, with a better grasp of these things than he has, can extend the language where it needs extending. This appendix is the deed to that gate: how an extension is made, and what it may never do.

The one gate

There is exactly one gate in Wuxia, and it is ∇.
Data becomes execution only through ∇, and nowhere else. That single fact is sometimes mistaken for several gates, so it is worth unpicking. Assembly is not a gate: it is data getting its shape, a train or a vector laid out where ∇ can reach it. Installation is not a gate: binding a name is an ordinary act, performed by an ordinary train. A trapdoor body between its two nablas is not a gate either: the nablas are the same boundary, with foreign rules inside it. Multiple gates would be illogical, because there is only one crossing to guard: the crossing from data to running code, and ∇ stands on it alone.
Everything else in this appendix is a consequence of that. Text is never executed as text, in any language: Dyalog's ⍎ is refused, and Python source supplied as data would be refused the same way, not by a second gate but by the same one. What is executed is always an assembled thing, named and defined, run under ∇ by whoever holds it.

Glyph assignment

The unassigned codepoints are the frontier. An extension begins by assigning one: every verb gets its own Unicode glyph and its own definition, recorded in the dictionary like any other verb. There is no numbered escape hatch, no I-beam: the APL ⌶ was considered and closed, because a number is not a name and a backdoor with no nameplate is how languages rot.

Style binds the extender as it bound the author. The more powerful verbs get the weirder letters. Bold fraktur capitals signal entire subsystems, not ordinary verbs, so a fraktur capital is a claim that a whole subsystem stands behind the glyph. And the sugar principle governs every assignment: the glyph should make the concept almost mindless to remember and use, because a glyph that must be looked up twice has already failed once.

The Trapdoor

Some holes are not verbs waiting for glyphs. They are speed, or I/O, or a library the engine already carries, and the honest way through them is down: the Trapdoor ⊆ U+1D21D.

The installer binds the extension at install time:

```
⇔ X ⊆ ∇ body ∇
```

X receives the address of the compiled body; once bound, niladic X calls the code, and ε maps to the accumulator, as the dictionary records. Between the nablas, different rules apply. The body is a foreign block, by assembly convention: a language word and braces,

```
asm { ... } python { ... } c { ... }
```

and inside the braces the named language's rules govern, not Wuxia's. Assembly serves speed and I/O, the work nothing else can reach. Python serves the semantic holes: the formats and protocols and databases the engine's libraries already know. C stands between them. In every case the discipline is the same: the body is written in source by the extender, bound under a name, and given a definition in the dictionary. A trapdoor binding is a verb in every way that matters, and it is recorded like one.

A trapdoor verb is engine-bound by nature: an asm body runs on the machine it was assembled for, a python body on the Python engine. That is not a flaw to apologize for. It is what a trapdoor is, and the dictionary entry says which floor it opens onto.

What an extension may not do

An extension may not mint a second gate. It may not execute text as text, in Wuxia or in a foreign body fed from data. It may not take a numbered or generic escape in place of a named glyph. And it may not reach silently: what a train carries over a pipe is data, and data installs nothing and executes nothing until someone holding ∇ runs it, which is that holder's decision and nobody else's.

Within those bounds, the bounds are the whole of the law, and they are few on purpose. The language was built to be extended by its betters. Extend it in the open, under a glyph, with a definition, through the one gate.

Index

English Index

AcidBroth 204-1
AcidBroth 234-3
AirElemental 214-3
Amulet 188-1
Anagram 76-4
Anagram 84-4
Angle 184-2
Angle 251-2
Anticut 152-1
Antilog 28-2
Apple 206-1
Apple 245-4
Applyself 132-3
Approx 13-2
Arccosine 26-1
Arcsine 26-2
Arctangent 26-4
Armor 187-3
Asian 198-4
Asian 239-4
Atomize 119-2
BabyDragon 226-3
BabyLongWorm 215-4
BabyPurpleWorm 215-2
Backstroke 116-1
Backstroke 122-1
Backstroke (forward) 116-3
Backstroke (forward) 122-3
Baluchitherium 228-2
Banana 207-1
Banana 246-4
Base Ten Log 29-4
Bell 39-3
Bell 44-3
Bell 47-1
BowlOfRice 200-1
BowlOfRice 241-1
Box 93-1
Box 95-1
Box 97-1
Box 101-1
Bucket 111-1
Bugbear 229-2
Caboose 119-4
Candy 202-4
Candy 248-3
CandyBar 209-4
CandyBar 243-1
CantoneseDuck 201-4
CantoneseDuck 242-4
Cardinality 82-1
CarpSashimi 196-1
CarpSashimi 237-1
Carrot 207-2
Carrot 247-1
Catalogue 95-3
Catalogue 99-3
Catenate 74-2
Catenate 80-1
Catenate 87-2
CaveSpider 227-1
Ceiling 25-1
Ceiling 35-2
Cell 178-1
Center 82-2
Centipede 226-1
ChickenNoodleSoup 205-4
ChickenNoodleSoup 236-3
Class 181-1
Classes 181-2
Clock 130-2
CloveOfGarlic 211-1
CloveOfGarlic 244-1
Collate 143-1
Combinatoric 51-3
Combinatoric 54-1

Comment 117-2
Compress 148-2
Compress 154-2
Congruent 9-3
Congruent 11-3
Conjugate 21-3
Constant 159-1
Cosine 25-4
Count 77-3
CrabSashimi 197-2
CrabSashimi 238-2
CramRation 210-2
CramRation 243-3
CRation 211-4
CRation 244-4
CreamOfAlgaeSoup 205-3
CreamOfAlgaeSoup 236-2
Creature 202-3
Creature 251-4
CrowBar 204-4
CrowBar 249-4
Curry 143-2
Cut 151-1
Cut 154-3
D10 42-2
D12 42-3
D2 41-1
D4 41-2
D6 41-3
Deal 45-1
Debugger 151-2
Decide 137-3
Decode 54-3
Decrement 163-2
Decrement 166-2
Deli 201-2
Deli 242-1
Depth 79-2
Determinant 68-1
Dice 41-4
Dice 44-4
Dice 47-2
Dice Percentile 42-1
Diction 137-1
Differentiate 58-3
Divide (Complex) 18-3
Divide (Complex) 23-3
Divide (Complex) 33-3
Divide (Polar) 18-4
Divide (Polar) 23-4
Divide (Polar) 33-4
Divide By 164-1
Divide By 167-1
Divide Matrix 70-4
Divisible 50-1
Divisible 53-2
Double 21-4
DoveBar 203-4
DoveBar 249-3
Down 178-3
Dragon 226-2
DragonSashimi 197-3
DragonSashimi 238-3
DRation 203-3
DRation 249-2
Drop 85-2
Drop Train 129-4
Dumplings 199-4
Dumplings 240-4
Dungeon 177-2
Duplicate 119-3
Duplicate Head 122-2
Duplicate Tail 124-3
DwarfMummy 221-1
DwarfZombie 223-2
EarthElemental 214-1
EelSashimi 196-3
EelSashimi 237-3
Egg 202-1
Egg 251-1
Egg 252-1
Elf 225-3
ElfMummy 220-2
ElfNoble 225-4

ElfZombie 223-3
ElvenMonarch 226-4
Encode 54-2
Enlist 108-1
Enumerate 77-1
Epsilon 159-2
Equality 12-1
Errdmd 151-3
Errline 148-3
Errmsg 148-4
Ettin 218-4
EttinMummy 222-2
EttinZombie 224-3
EucalyptusLeaf 209-3
EucalyptusLeaf 242-3
Euler 40-4
Evaluate 61-4
Expand 87-4
Extend 74-1
Extend 81-2
Extend 87-1
Exx 73-3
Factor 51-2
Factorial 19-1
Factorial 29-1
Factorial 35-1
Fetch 99-2
Fig 207-3
Fig 247-2
Find 110-4
FireElemental 214-2
FireGiant 218-1
FixedPoint 62-1
Floor 25-2
Floor 35-3
Foocubus 231-3
Food 188-2
FoodRation 210-3
FoodRation 243-4
forall 14-1
Fork 130-1
Format 140-4
Format 143-3
Fornone 14-3
Forsome 14-2
FortuneCookie 199-2
FortuneCookie 240-2
Fourier 29-2
FrostGiant 218-3
Fruit 201-3
Fruit 245-3
Function 137-4
Gamma 40-1
GCD 53-3
Gem 189-1
Gets 167-3
Ghoul 224-1
Giant 217-2
GiantMummy 222-1
GiantSpider 227-3
GiantZombie 224-2
Glean 87-3
Glyph 137-2
GnomeMummy 221-2
GnomeZombie 222-4
Golden 40-3
GoldenSoup 204-2
GoldenSoup 235-1
Goodbar 203-1
Goodbar 248-2
Grade Down 74-4
Grade Up 74-3
GrandMaster 234-4
Grapes 208-4
Grapes 248-4
Greater Or Equal 12-4
Greater Than 12-2
GreenElf 225-1
GreyElf 225-2
Gyuniku 198-3
Gyuniku 239-3
Half 39-1
Half 44-1
Head 124-1

HillGiant 218-2
Honee 203-2
Honee 249-1
HumanMummy 220-1
HumanZombie 223-4
Hyperbolic Arccosine 28-4
Hyperbolic Arcsine 27-3
Hyperbolic Arctangent 28-3
Hyperbolic Cosine 27-2
Hyperbolic Sine 27-1
Hyperbolic Tangent 27-4
Identity 115-4
Implies 13-1
In 110-1
In 112-1
InAt 110-3
Increment 163-1
Increment 166-1
Indivisible 50-2
Indivisible 53-4
Inherits 183-2
InLast 110-2
Inner Product 65-2
Inner Product 67-2
Inner Product 70-2
InOf 107-3
Instance 181-4
Instances 181-3
Integer Division 61-3
Integrate 58-4
Interpreter 183-1
Intersection 105-1
Intersection 107-1
Intersection 111-2
InverseFourier 29-3
Invert 9-1
Isa 183-3
IsAt 11-4
Jabberwock 230-1
Kelp 199-3
Kelp 240-3
KelpFrond 209-2
KelpFrond 242-2
Keyboard 117-1
Kobold 216-2
KoboldLord 217-3
KoboldMummy 221-4
KoboldShaman 217-4
KoboldZombie 222-3
Kount 77-4
KRation 210-4
KRation 244-3
L Factor 67-4
Lambda 141-1
Laminate 76-3
Laminate 84-3
LargeKobold 216-1
Lay 140-3
LCM 53-1
Left 128-3
Left 132-2
LembasWafer 210-1
LembasWafer 243-2
Lemon 207-4
Lemon 247-3
Leocrotta 230-3
Less Or Equal 12-3
Less Than 11-1
Level 177-4
Lime 208-3
Lime 247-4
Link 99-4
(literalize) 139-4
Local 59-3
LongWorm 215-3
LumpOfRoyalJelly 211-3
LumpOfRoyalJelly 245-2
LurkerAbove 230-4
Magnitude 30-1
Manwich 205-2
Manwich 235-4
Map 115-2
Map 121-2
MasterMindFlayer 216-3

Mastodon 227-4
Match 11-2
Meat 202-2
Meat 250-4
Meat 252-4
Melon 206-4
Melon 246-3
MerfolkCommon 233-3
MerfolkNoble 233-4
MerfolkRoyal 232-4
MerfolkWitch 233-2
Meter 119-1
(method binding) 167-2
MindFlayer 216-4
Minotaur 228-3
Minus (Complex) 18-1
Minus (Complex) 23-1
Minus (Complex) 33-1
Mix 79-1
MoldySoup 205-1
MoldySoup 235-3
Monster 192-2
MountainNymph 232-1
Multiply 19-4
Multiply 24-4
Multiply 34-4
Multiply By 163-3
Multiply By 166-3
Multiply Matrices 65-3
Multiply Matrices 67-3
Multiply Matrices 70-3
Mumak 228-1
Natural Log 28-1
Natural Log 35-4
Negate 21-2
Negate / Subtract 17-2
NextLocal 58-1
Nil 7-2
Nip 128-1
Nosferatu 220-4
Not 9-2
Object 183-4
Omega 159-3
Optimize 59-2
Optimize 61-2
Orange 206-2
Orange 246-1
OrcMummy 221-3
OrcZombie 223-1
Outer Product 65-1
Outer Product 67-1
Outer Product 70-1
Over 128-4
Owlbear 229-3
Oyster 198-1
Oyster 239-1
Partition 101-3
Pear 206-3
Pear 246-2
Pee 73-1
Percent 19-2
Percent 24-1
Percent 34-1
Permute 76-2
Permute 84-2
Person 159-4
Pesto 209-1
Pesto 250-3
Pi 40-2
Pick 42-4
Pick 44-2
Pipeline 122-4
Pipeline 132-4
Player 192-1
Plus 32-1
Plus (Complex) 17-3
Plus (Complex) 22-3
Plus (Complex) 32-3
Point 171-1
Potion 189-2
PreservedEgg 200-4
PreservedEgg 241-4
Prim 170-3
Prime 50-4

Prime 51-1
Primes 50-3
Print 140-1
Property 163-4
Property 166-4
PurpleWorm 215-1
Put 84-1
Quote Double 139-3
Quote Single 139-2
Quote Triple 139-1
Quotient 24-3
Quotient 34-3
Rail 140-2
Ramen 199-1
Ramen 240-1
Rank 76-1
Raster 170-4
Ravel 80-4
RavelRows 80-3
Raze 95-4
Real 39-4
Reduce 115-1
Reduce 121-1
Reduce (forward) 115-3
Reduce (forward) 121-3
Reflect 77-2
RefriedBeans 195-2
RefriedBeans 236-1
Replicate 126-2
Reveal 61-1
Reverse 79-3
Reverse 85-3
Reverse Train 129-3
Right 128-2
Right 132-1
Ring 187-1
Roots 58-2
Rotate 81-4
Rotate 90-2
Rotate Head 126-3
Rotate Tail 126-4
Rothe 230-2
SalmonSashimi 196-4
SalmonSashimi 237-4
Scorpion 227-2
Screen 177-3
Scroll 188-3
Set Difference 111-4
Shade 231-1
Shape 81-1
Shape 90-1
Shopkeeper 233-1
Sight 81-3
Signum 21-1
Signum / Plus 17-1
Silence 148-1
Silence 154-1
Sine 25-3
Skeleton 231-2
Solve 59-1
Soup 201-1
Soup 248-1
Spinach 195-3
Spinach 250-1
Split 80-2
SprigOfWolfsbane 211-2
SprigOfWolfsbane 244-2
SpringRolls 200-2
SpringRolls 241-2
Square 19-3
Square 36-1
Square Root 22-2
Stalker 231-4
Star 24-2
Star 34-2
StonedSoup 204-3
StonedSoup 235-2
StoneGiant 217-1
Stop 116-2
StormGiant 219-2
Strip 116-4
Strip 124-4
Subtract 32-2
Succeed 151-4

Sum 22-1
Summarize 93-3
Summarize 97-3
Summarize 99-1
Sundry 208-2
Sundry 245-1
Sushi 195-4
Sushi 236-4
Swap 126-1
Tail 124-2
Take 85-1
Tangent 26-3
Temporary Vector 161-1
Time 39-2
Times (Complex) 17-4
Times (Complex) 22-4
Times (Complex) 32-4
Times (Polar) 18-2
Times (Polar) 23-2
Times (Polar) 33-2
Tin 195-1
Tin 234-2
Titan 219-4
Titanothere 228-4
Tofu 200-3
Tofu 241-3
Tomb 208-1
Tomb 250-2
Tool 188-4
Transpose 79-4
Transpose 85-4
Trapdoor 170-1
Trapdoor 173-1
Trapper 229-4
Triangle 171-2
Triple 129-1
Truth 7-3
Tuck 129-2
TunaSashimi 197-1
TunaSashimi 238-1
TurtleSashimi 197-4
TurtleSashimi 238-4
U Factor 68-2
UnApprox 13-3
Unbox 93-2
Unbox 97-2
Unbox 101-2
Unequal 13-4
Uni 198-2
Uni 239-2
Union 105-2
Union 107-2
Union 111-3
Unquote Double 135-3
Unquote Single 135-2
Unquote Triple 135-1
Up 178-2
Vampire 219-3
VampireBat 219-1
VampireLeader 220-3
VirtualDistance 184-1
VirtualDistance 252-2
VirtualHorizontal 184-3
VirtualHorizontal 251-3
VirtualSpace 170-2
VirtualVertical 184-4
VirtualVertical 252-3
Wand 187-2
WaterElemental 214-4
WaterNymph 232-3
Weapon 187-4
WhaleSashimi 196-2
WhaleSashimi 237-2
Where 107-4
Where 177-1
WizardOfYendor 234-1
WoodElf 224-4
WoodNymph 232-2
Wumpus 229-1
Wuxia 7-1
Wye 73-4
Zee 73-2
Zero 121-4
Zip 95-2

Glyph Index

!	(U+0021)	19-1, 29-1, 35-1
%	(U+0025)	19-2, 24-1, 34-1
&	(U+0026)	59-1
*	(U+002A)	24-2, 34-2
+	(U+002B)	17-1, 21-1, 32-1
-	(U+002D)	17-2, 21-2, 32-2
/	(U+002F)	115-1, 121-1
<	(U+003C)	11-1
=	(U+003D)	12-1
>	(U+003E)	12-2
?	(U+003F)	45-1
@	(U+0040)	177-1, 230-1, 231-2, 231-3, 232-1 232-2, 232-3
\	(U+005C)	116-1, 122-1
~	(U+007E)	9-1
¡	(U+00A1)	137-1
¢	(U+00A2)	178-1, 226-1
£	(U+00A3)	177-2
¤	(U+00A4)	227-1
¥	(U+00A5)	234-1
§	(U+00A7)	177-3, 233-1
«	(U+00AB)	139-1
¬	(U+00AC)	9-2
®	(U+00AE)	137-2
±	(U+00B1)	143-1, 227-2
¶	(U+00B6)	177-4
»	(U+00BB)	135-1
½	(U+00BD)	39-1, 44-1
¿	(U+00BF)	137-3, 229-1
Ø	(U+00D8)	227-3
ö	(U+00F6)	229-2
÷	(U+00F7)	24-3, 34-3
ø	(U+00F8)	178-1, 229-3, 251-1, 252-1
Ĝ	(U+011C)	218-1
Ğ	(U+011E)	218-2
Ġ	(U+0120)	217-1
Ģ	(U+0122)	217-2
ƀ	(U+0180)	219-1
Ɠ	(U+0193)	218-3
Ƹ	(U+01B8)	230-1
Ǥ	(U+01E4)	219-2
Ǧ	(U+01E6)	218-4
Ȣ	(U+0222)	230-2
Ƚ	(U+023D)	230-3
Δ	(U+0394)	226-2
Λ	(U+039B)	141-1
Ψ	(U+03A8)	227-4, 228-1
α	(U+03B1)	159-1
γ	(U+03B3)	40-1
δ	(U+03B4)	226-3
ε	(U+03B5)	159-2
π	(U+03C0)	40-2
ρ	(U+03C1)	81-1, 90-1
φ	(U+03C6)	40-3
ω	(U+03C9)	159-3
ό	(U+03CC)	233-2
ϐ	(U+03D0)	232-4
ϒ	(U+03D2)	228-2
Ϝ	(U+03DC)	29-2
Ќ	(U+040C)	217-3
Э	(U+042D)	231-4
Ѫ	(U+046A)	220-1, 220-2, 221-1, 221-2, 221-3 221-4, 222-1, 222-2
Й̨	(U+048A)	229-4
ҟ	(U+049B)	217-4
Ҝ	(U+049C)	216-1
ҝ	(U+049D)	216-2
Ӌ	(U+04F4)	216-3
ӌ	(U+04F5)	216-4
ᵷ	(U+1D7F)	161-1
Ṁ	(U+1E40)	228-3
ṽ	(U+1E7D)	220-3
ẏ	(U+1E7F)	219-3
ȯ	(U+1F40)	233-3
ȯ	(U+1F42)	233-4
‘	(U+2018)	139-2

’	(U+2019)	135-2
“	(U+201C)	139-3
”	(U+201D)	135-3
‰	(U+2030)	42-1
⁉	(U+203D)	183-1
ℇ	(U+2107)	40-4
ℊ	(U+210A)	224-1
ℓ	(U+2113)	58-1
№	(U+2116)	77-1
ℙ	(U+2119)	73-1
ℜ	(U+211C)	61-1
ℝ	(U+211D)	187-1
℞	(U+211E)	58-2
℣	(U+2123)	220-4
ℤ	(U+2124)	73-2, 222-3, 222-4, 223-1, 223-2, 223-3, 223-4, 224-2, 224-3
℩	(U+2129)	29-3
ℭ	(U+212D)	228-4
ℾ	(U+213E)	74-1, 81-2, 87-1
ℿ	(U+213F)	74-2, 80-1, 87-2
⅁	(U+2141)	219-4
⅋	(U+214B)	59-2, 61-2
⅏	(U+214F)	230-4
Ⅱ	(U+2161)	122-2
Ⅲ	(U+2162)	129-1
↑	(U+2191)	85-1
↓	(U+2193)	85-2
⇋	(U+21CB)	126-1
⇑	(U+21D1)	79-1
⇒	(U+21D2)	13-1
⇓	(U+21D3)	80-2
↤	(U+21F7)	128-1
↦	(U+21F8)	129-2
↮	(U+21F9)	129-3
∀	(U+2200)	14-1
∂	(U+2202)	58-3
∃	(U+2203)	14-2
∄	(U+2204)	14-3
∅	(U+2205)	7-2
∇	(U+2207)	137-4
∈	(U+2208)	110-1
ϵ	(U+220A)	108-1, 112-1
∐	(U+2210)	82-1, 111-1
∑	(U+2211)	22-1
∔	(U+2214)	119-1
√	(U+221A)	22-2
∩	(U+2229)	105-1, 107-1, 111-2
∪	(U+222A)	105-2, 107-2, 111-3
∫	(U+222B)	58-4
∷	(U+2237)	119-2
∻	(U+223B)	77-2
≈	(U+2248)	13-2
≉	(U+2249)	13-3
≖	(U+2256)	95-1
≙	(U+2257)	11-2
≠	(U+2260)	13-4
≡	(U+2261)	9-3, 11-3
≣	(U+2263)	11-4
≤	(U+2264)	12-3
≥	(U+2265)	12-4
≬	(U+226C)	115-2, 121-2
⊂	(U+2282)	93-1, 97-1, 101-1
⊃	(U+2283)	93-2, 97-2, 101-2
⊆	(U+2286)	101-3
⊍	(U+228D)	79-2
⊑	(U+2291)	183-2
⊕	(U+2295)	163-1, 166-1, 214-1
⊖	(U+2296)	163-2, 166-2
⊗	(U+2297)	65-1, 67-1, 70-1
⊘	(U+2298)	164-1, 167-1
⊙	(U+2299)	181-1
⊚	(U+229A)	93-3, 97-3, 99-1
⊛	(U+229B)	163-3, 166-3, 214-2
⊞	(U+229E)	82-2
⊠	(U+22A0)	19-3, 36-1
⊡	(U+22A1)	167-2
⊢	(U+22A2)	128-2, 132-1
⊣	(U+22A3)	128-3, 132-2
⊤	(U+22A4)	7-3, 215-1
⊦	(U+22A6)	215-2
⊰	(U+22B0)	95-2
⊶	(U+22B6)	21-3
⊷	(U+22B7)	119-3
⊸	(U+22B8)	99-2

⊹	(U+22B9)	148-1, 154-1
⋄	(U+22C4)	116-2
⋅	(U+22C5)	65-2, 67-2, 70-2
⋈	(U+22C8)	65-3, 67-3, 70-3
⋔	(U+22D4)	130-1
⋕	(U+22D5)	76-1
⋖	(U+22D6)	148-2, 154-2
⋘	(U+22D8)	163-4, 166-4
⋮	(U+22EE)	151-1, 154-3
⋯	(U+22EF)	152-1
⋲	(U+22F2)	80-3
⋳	(U+22F3)	80-4
⋵	(U+22F5)	110-2
⋶	(U+22F8)	107-3
⋹	(U+22F9)	110-3
⋿	(U+22FF)	183-3
⌈	(U+2308)	25-1, 35-2
⌊	(U+230A)	25-2, 35-3
⌚	(U+231A)	39-2
⌨	(U+2328)	117-1
⌬	(U+232C)	126-2
⌹	(U+2339)	70-4
⍃	(U+233C)	68-1
⍎	(U+233D)	79-3, 85-3
⍏	(U+233F)	115-3, 121-3
⍜	(U+2340)	116-3, 122-3
⍉	(U+2349)	79-4, 85-4
⍋	(U+234B)	74-3
⍒	(U+2352)	74-4
⍗	(U+2357)	30-1
⍰	(U+235B)	18-1, 23-1, 33-1
⍠	(U+2360)	140-1
⍣	(U+2363)	132-3
⍳	(U+2373)	77-3
⍷	(U+2377)	110-4
⎷	(U+237C)	28-1, 35-4
⎈	(U+2388)	81-3
⎊	(U+238A)	53-1
⎢	(U+23A2)	50-1, 53-2
⏁	(U+23C1)	51-1
⏱	(U+23F1)	130-2
⑩	(U+2469)	42-2
⑫	(U+246B)	42-3
⑳	(U+2473)	201-1, 248-1
Ⓒ	(U+24B8)	181-2
Ⓘ	(U+24BE)	181-3
▲	(U+25B2)	178-2
▼	(U+25BC)	178-3
●	(U+25CF)	189-1
◝	(U+25DD)	25-3
◞	(U+25DE)	25-4
◶	(U+25F6)	26-1
◷	(U+25F7)	26-2
☘	(U+2618)	223-4, 225-1, 225-2, 225-3
☻	(U+263B)	192-1
⚁	(U+2681)	41-1
⚃	(U+2683)	41-2
⚅	(U+2685)	41-3
✌	(U+270C)	117-2
✔	(U+2714)	151-2
✧	(U+2727)	151-3
✪	(U+272A)	195-1, 234-2
❁	(U+2741)	188-1
❌	(U+274C)	148-3
❎	(U+274E)	148-4
❖	(U+2756)	201-2, 242-1
❗	(U+2757)	189-2
❶	(U+2776)	204-1, 234-3
❷	(U+2777)	204-2, 235-1
❸	(U+2778)	204-3, 235-2
❹	(U+2779)	205-1, 235-3
❺	(U+277A)	205-2, 235-4
❻	(U+277B)	195-2, 236-1
❼	(U+277C)	195-3, 250-1
❾	(U+277E)	205-3, 236-2
❿	(U+277F)	205-4, 236-3
⟒	(U+27D2)	51-2
⟗	(U+27D7)	87-3
⟪	(U+27DA)	84-1
⟰	(U+27F0)	184-1, 252-2
⟲	(U+27F2)	126-3
⟳	(U+27F3)	126-4
⤚	(U+291A)	129-4
⤺	(U+293A)	128-4

 (U+2980) 53-3
 (U+2991) 124-1
 (U+2992) 124-2
 (U+2999) 151-4
 (U+29B5) 184-2, 251-2
 (U+29BD) 81-4, 90-2
 (U+29CA) 28-2
 (U+29D3) 184-3, 251-3
 (U+29D7) 184-4, 252-3
 (U+29DE) 124-3
 (U+2A02) 18-2, 23-2, 33-2
 (U+2A03) 42-4, 44-2
 (U+2A0A) 76-2, 84-2
 (U+2A21) 187-2
 (U+2A22) 17-3, 22-3, 32-3
 (U+2A23) 77-4
 (U+2A27) 21-4
 (U+2A2C) 18-3, 23-3, 33-3
 (U+2A2F) 19-4, 24-4, 34-4
 (U+2A30) 17-4, 22-4, 32-4
 (U+2A33) 139-4
 (U+2A37) 95-3, 99-3, 214-3
 (U+2A38) 18-4, 23-4, 33-4, 214-4
 (U+2A42) 107-4
 (U+2A43) 111-4
 (U+2A4D) 28-3
 (U+2A53) 215-3
 (U+2A55) 215-4
 (U+2A5E) 76-3, 84-3
 (U+2A5F) 29-4
 (U+2A69) 51-3, 54-1
 (U+2A78) 119-4
 (U+2AA6) 27-1
 (U+2AA7) 27-2
 (U+2ABD) 27-3
 (U+2ABE) 28-4
 (U+2ACD) 140-2
 (U+2ACE) 140-3
 (U+2ACF) 116-4, 124-4
 (U+2AD1) 27-4
 (U+2AD7) 26-3
 (U+2AD8) 26-4
 (U+2AE7) 54-2
 (U+2AE8) 54-3
 (U+2AE9) 99-4
 (U+2AEE) 50-2, 53-4
 (U+2AF5) 95-4
 (U+2AF7) 122-4, 132-4
 (U+2AFD) 61-3
 (U+2B04) 167-3
 (U+2E19) 225-4
 (U+2F40) 226-4
 (U+3248) 208-1, 250-2
 (U+3249) 209-1, 250-3
 (U+3267) 208-2, 245-1
 (U+3280) 196-1, 237-1
 (U+3281) 196-2, 237-2
 (U+3282) 196-3, 237-3
 (U+3283) 196-4, 237-4
 (U+3284) 197-1, 238-1
 (U+3285) 197-2, 238-2
 (U+3286) 197-3, 238-3
 (U+3287) 197-4, 238-4
 (U+3288) 198-1, 239-1
 (U+3289) 198-2, 239-2
 (U+328A) 198-3, 239-3
 (U+328C) 209-2, 242-2
 (U+328D) 209-3, 242-3
 (U+328E) 209-4, 243-1
 (U+328F) 210-1, 243-2
 (U+3290) 210-2, 243-3
 (U+3297) 195-4, 236-4
 (U+329F) 211-1, 244-1
 (U+32A0) 211-2, 244-2
 (U+32A2) 211-3, 245-2
 (U+32A9) 210-3, 243-4
 (U+32AA) 210-4, 244-3
 (U+32AB) 211-4, 244-4
 (U+4E3D) 202-2, 250-4, 252-4
 (U+4EBA) 159-4
 (U+4FA0) 7-1
 (U+7236) 234-4
 (U+A74B) 202-3, 251-4
 (U+AB44) 62-1

𝔊	(U+1D21D)	170-1, 173-1	𝟚	(U+1D7DA)	171-1
𝒜	(U+1D49C)	76-4, 84-4	𝟛	(U+1D7DB)	171-2
𝒞	(U+1D49E)	143-2	🀀	(U+1F000)	203-1, 248-2
𝒫	(U+1D4AB)	50-3	🀁	(U+1F001)	203-2, 249-1
𝒱	(U+1D4E5)	170-2	🀂	(U+1F002)	203-3, 249-2
𝔏	(U+1D50F)	59-3	🀃	(U+1F003)	203-4, 249-3
𝔓	(U+1D513)	50-4	🀩	(U+1F029)	204-4, 249-4
𝔸	(U+1D538)	187-3	🀪	(U+1F02A)	202-4, 248-3
𝔽	(U+1D53D)	188-2	🁰	(U+1F030)	201-3, 245-3
𝕀	(U+1D540)	181-4	🁲	(U+1F032)	206-1, 245-4
𝕄	(U+1D544)	192-2	🁳	(U+1F033)	206-2, 246-1
𝕆	(U+1D546)	183-4	🁴	(U+1F034)	206-3, 246-2
𝕊	(U+1D54A)	188-3	🁵	(U+1F035)	206-4, 246-3
𝕋	(U+1D54B)	188-4	🁶	(U+1F036)	207-1, 246-4
𝕎	(U+1D54E)	187-4	🁷	(U+1F037)	207-2, 247-1
𝕏	(U+1D54F)	73-3	🁸	(U+1F038)	207-3, 247-2
𝕐	(U+1D550)	73-4	🁹	(U+1F039)	207-4, 247-3
𝔅	(U+1D56D)	39-3, 44-3, 47-1	🁺	(U+1F03A)	208-3, 247-4
𝔇	(U+1D56F)	41-4, 44-4, 47-2	🁻	(U+1F03B)	208-4, 248-4
𝔈	(U+1D570)	87-4	🉐	(U+1F210)	198-4, 239-4
𝔉	(U+1D571)	140-4, 143-3	🉑	(U+1F211)	199-1, 240-1
𝔍	(U+1D574)	115-4	🉒	(U+1F212)	199-2, 240-2
𝔏	(U+1D577)	67-4	🉓	(U+1F213)	199-3, 240-3
𝔑	(U+1D57D)	39-4	🉔	(U+1F214)	199-4, 240-4
𝔘	(U+1D580)	68-2	🉕	(U+1F215)	200-1, 241-1
𝔝	(U+1D585)	121-4	🉖	(U+1F216)	200-2, 241-2
𝔣	(U+1D58B)	61-4	🉗	(U+1F217)	200-3, 241-3
𝟘	(U+1D7D8)	170-3	🉘	(U+1F218)	200-4, 241-4
𝟙	(U+1D7D9)	170-4	🉙	(U+1F219)	201-4, 242-4